

Human review packet: GHW01DigitalGoods

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GHW01DigitalGoods
Paper id: GHW01DigitalGoods
Source version: GHW01 byte-pinned source inventory
Source record: audit/paper_statement_map.json
Rows: 12
Generated: 2026-09-08
Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GHW01DigitalGoods --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(97; supporting context\)](#)
- [Paper-specific semantic prerequisites \(73; not paper claims\)](#)
 - [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
 - [GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeLaw](#)
 - [GHW01DigitalGoods.ProofBridge.source_weightedPairingAuction](#)
 - [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue](#)
 - [GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeExpectedRevenue](#)
 - [GHW01DigitalGoods.ProofBridge.source_weightedPairingOutcomeExpectedRevenue](#)
 - [GHW01DigitalGoods.ProofBridge.source_weightedPairingW](#)
 - [GHW01DigitalGoods.ProofBridge.totalValue](#)
 - [GHW01DigitalGoods.SourceDefinitions.AdmissibleVickreySupplyRule](#)
 - [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
 - [GHW01DigitalGoods.SourceDefinitions.BidValues](#)
 - [GHW01DigitalGoods.SourceDefinitions.AuctionInput](#)
 - [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.FillsAtOrBelow](#)
 - [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.IsSinglePrice](#)
 - [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.profit](#)
 - [GHW01DigitalGoods.SourceDefinitions.BidIndependentRule](#)
 - [GHW01DigitalGoods.SourceDefinitions.CompetitiveOn](#)
 - [GHW01DigitalGoods.SourceDefinitions.DeterministicAuction](#)
 - [GHW01DigitalGoods.SourceDefinitions.DeterministicTruthful](#)
 - [GHW01DigitalGoods.SourceDefinitions.DeterministicCorrect](#)
 - [GHW01DigitalGoods.SourceDefinitions.DeterministicIsSinglePrice](#)
 - [GHW01DigitalGoods.SourceDefinitions.boundedSupplyFk](#)
 - [GHW01DigitalGoods.SourceDefinitions.IsBoundedSupplyOptimalThreshold](#)
 - [GHW01DigitalGoods.SourceDefinitions.IsKItemBoundedSupplyAuction](#)
 - [GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyOutcome](#)

- GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyAuction
- GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue
- GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue
- GHW01DigitalGoods.SourceDefinitions.IsOptimalThreshold
- GHW01DigitalGoods.SourceDefinitions.lowestBid
- GHW01DigitalGoods.SourceDefinitions.NormalizedBidScale
- GHW01DigitalGoods.SourceDefinitions.PositiveAlpha
- GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.directionalRealization
- GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.directionalOutcomeLaw
- GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.dualPriceRealization
- GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.dualPriceOutcomeLaw
- GHW01DigitalGoods.SourceDefinitions.RandomizedAuction
- GHW01DigitalGoods.SourceDefinitions.RandomizedTruthful
- GHW01DigitalGoods.SourceDefinitions.RandomizedCorrect
- GHW01DigitalGoods.SourceDefinitions.RandomizedIsSinglePrice
- GHW01DigitalGoods.SourceDefinitions.RevenueBigO
- GHW01DigitalGoods.SourceDefinitions.RevenueBigOmega
- GHW01DigitalGoods.SourceDefinitions.SortedBids
- GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary
- GHW01DigitalGoods.SourceDefinitions.TruthfulReports
- GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyDeterministicAuction
- GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyRandomizedAuction
- GHW01DigitalGoods.SourceDefinitions.accepts
- GHW01DigitalGoods.SourceDefinitions.bidIndependentOutcome
- GHW01DigitalGoods.SourceDefinitions.bidIndependentAuction
- GHW01DigitalGoods.SourceDefinitions.bidderCount
- GHW01DigitalGoods.SourceDefinitions.boundedDualPriceSampling
- GHW01DigitalGoods.SourceDefinitions.boundedDualExactHalfOutcomeLaw
- GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceFixedSizeOutcomeLaw
- GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceSampling
- GHW01DigitalGoods.SourceDefinitions.boundedSupplyOptK
- GHW01DigitalGoods.SourceDefinitions.boundedSupplyTk
- GHW01DigitalGoods.SourceDefinitions.deterministicOptimalThresholdAuction
- GHW01DigitalGoods.SourceDefinitions.deterministicRevenue
- GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdAuction
- GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdOutcomeLaw
- GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome
- GHW01DigitalGoods.SourceDefinitions.fixedPriceAuction
- GHW01DigitalGoods.SourceDefinitions.fixedPriceScaleCondition
- GHW01DigitalGoods.SourceDefinitions.highestBid
- GHW01DigitalGoods.SourceDefinitions.independentHalfSamplingAnalysisLaw
- GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceAuction
- GHW01DigitalGoods.SourceDefinitions.optimalThreshold
- GHW01DigitalGoods.SourceDefinitions.performanceRatio
- GHW01DigitalGoods.SourceDefinitions.randomizedRevenueLaw
- GHW01DigitalGoods.SourceDefinitions.scarceSupply
- GHW01DigitalGoods.SourceDefinitions.totalBidValue
- GHW01DigitalGoods.SourceDefinitions.totalValueScaleCondition
- Source claims (12)
 - result_theorem4_1Spec
 - result_corollary4_2Spec
 - result_lemma6_1_fixed_size_lower_tailSpec
 - result_theorem6_2_random_samplingSpec
 - result_theorem7_1_weighted_pairingSpec
 - result_theorem7_2_lower_boundSpec
 - result_theorem7_2_tightnessSpec
 - result_lemma8_1_monotone_allocationSpec
 - result_theorem8_2_journal_revenue_upper_boundSpec
 - result_theorem9_1_bid_independent_lower_boundSpec
 - result_lemma9_2_bid_independenceSpec
 - result_theorem9_3_deterministic_lower_boundSpec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.Auction.DeterministicOfferBidIndependent

Declaration location: reusable library

Lean declaration body

```
type:
( $\mathbb{R} \rightarrow \text{Option } \mathbb{R}$ )  $\rightarrow$  Prop

value:
fun (offer :  $\mathbb{R} \rightarrow \text{Option } \mathbb{R}$ ) =>
  ( $\forall$  (report :  $\mathbb{R}$ ), offer report = none)  $\vee$ 
   $\exists$  (criticalPrice :  $\mathbb{R}$ ),
    ( $\forall$  (report price :  $\mathbb{R}$ ), offer report = some price  $\rightarrow$  price = criticalPrice)  $\wedge$ 
    ( $\forall$  report < criticalPrice, offer report = none)  $\wedge$ 
     $\forall$  (report :  $\mathbb{R}$ ), criticalPrice < report  $\rightarrow$  offer report = some criticalPrice
```

AppliedModelingLib.Auction.DigitalGoodsAuction

Declaration location: reusable library

Lean declaration body

```
field allocation:
{Agent : Type u_1}  $\rightarrow$  AppliedModelingLib.Auction.DigitalGoodsAuction Agent  $\rightarrow$  (Agent  $\rightarrow \mathbb{R}$ )  $\rightarrow$  Agent  $\rightarrow \mathbb{R}$ 

field payment:
{Agent : Type u_1}  $\rightarrow$  AppliedModelingLib.Auction.DigitalGoodsAuction Agent  $\rightarrow$  (Agent  $\rightarrow \mathbb{R}$ )  $\rightarrow$  Agent  $\rightarrow \mathbb{R}$ 
```

AppliedModelingLib.Auction.DigitalGoodsAuction.BinaryAllocation

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1}  $\rightarrow$  AppliedModelingLib.Auction.DigitalGoodsAuction Agent  $\rightarrow$  Prop

value:
fun {Agent : Type u_1} (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) =>
   $\forall$  (bids : Agent  $\rightarrow \mathbb{R}$ ) (i : Agent), M.allocation bids i = 0  $\vee$  M.allocation bids i = 1
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)

AppliedModelingLib.Auction.DigitalGoodsAuction.NoPositiveTransfers

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1}  $\rightarrow$  AppliedModelingLib.Auction.DigitalGoodsAuction Agent  $\rightarrow$  Prop

value:
fun {Agent : Type u_1} (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) =>
   $\forall$  (bids : Agent  $\rightarrow \mathbb{R}$ ) (i : Agent), 0  $\leq$  M.payment bids i
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)

AppliedModelingLib.Auction.DigitalGoodsAuction.utility

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → AppliedModelingLib.Auction.DigitalGoodsAuction Agent → (Agent → ℝ) → Agent → (Agent → ℝ) → ℝ

value:
fun {Agent : Type u_1} (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) (values : Agent → ℝ) (i : Agent)
  (bids : Agent → ℝ) =>
  values i * M.allocation bids i - M.payment bids i
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)

AppliedModelingLib.Auction.DigitalGoodsAuction.IndividuallyRational

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → AppliedModelingLib.Auction.DigitalGoodsAuction Agent → Prop

value:
fun {Agent : Type u_1} (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) =>
  ∀ (values : Agent → ℝ) (i : Agent), 0 ≤ M.utility values i values
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.utility](#)

AppliedModelingLib.Auction.GhwTightLowerAgent

Declaration location: paper-local

Lean declaration body

```
type:
ℕ → Type

value:
fun (k : ℕ) => (r : Fin (k - 1)) × Fin (2 ^ (k - (↑r + 1)))
```

AppliedModelingLib.Auction.GhwTightAgent

Declaration location: paper-local

Lean declaration body

```
type:
ℕ → ℕ → Type

value:
fun (k s : ℕ) => AppliedModelingLib.Auction.GhwTightLowerAgent k ⊕ Fin s
```

Direct retained dependencies

- [AppliedModelingLib.Auction.GhwTightLowerAgent](#)

AppliedModelingLib.Auction.TwoValueAgent

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℕ → Type

value:
fun (highCount lowCount : ℕ) => Fin highCount ⊕ Fin lowCount
```

AppliedModelingLib.Auction.deterministicAuctionOffer

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} →
[DecidableEq Agent] → AppliedModelingLib.Auction.DigitalGoodsAuction Agent → (Agent → ℝ) → Agent → ℝ → Option ℝ

value:
fun {Agent : Type u_1} [DecidableEq Agent] (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) (bids : Agent → ℝ)
  (i : Agent) (a : ℝ) =>
  have profile : Agent → ℝ := Function.update bids i a;
  if M.allocation profile i = 1 then some (M.payment profile i) else none
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)

AppliedModelingLib.Auction.finiteCandidatePriceSet

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → ℝ) → Finset ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → ℝ) => insert 0 (Finset.image values Finset.univ)
```

AppliedModelingLib.Auction.ghwTightValue

Declaration location: paper-local

Lean declaration body

```
type:
(k s : ℕ) → AppliedModelingLib.Auction.GhwTightAgent k s → ℝ

value:
fun (k s : ℕ) (a : AppliedModelingLib.Auction.GhwTightAgent k s) =>
  match a with
  | Sum.inl a => 2 ^ (↑a.fst + 1)
  | Sum.inr val => 2 ^ k
```

Direct retained dependencies

- [AppliedModelingLib.Auction.GhwTightAgent](#)
- [AppliedModelingLib.Auction.GhwTightLowerAgent](#)

AppliedModelingLib.Auction.paper_digital_goods_truthful

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → AppliedModelingLib.Auction.DigitalGoodsAuction Agent → Prop

value:
fun {Agent : Type u_1} [DecidableEq Agent] (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) =>
  ∀ (values : Agent → ℝ) (i : Agent) (report : ℝ),
    M.utility values i (Function.update values i report) ≤ M.utility values i values
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.utility](#)

AppliedModelingLib.Auction.paper_theorem9_3_erased_report_list

Declaration location: paper-local

Lean declaration body

```
type:
{highCount lowCount : ℕ} →
(AppliedModelingLib.Auction.TwoValueAgent highCount lowCount → ℝ) →
  AppliedModelingLib.Auction.TwoValueAgent highCount lowCount → List ℝ

value:
fun {highCount lowCount : ℕ} (bids : AppliedModelingLib.Auction.TwoValueAgent highCount lowCount → ℝ)
  (i : AppliedModelingLib.Auction.TwoValueAgent highCount lowCount) =>
  match i with
  | Sum.inl i =>
    (List.ofFn fun (j : Fin highCount) => bids (Sum.inl j)).eraseldx ↑i ++
    List.ofFn fun (j : Fin lowCount) => bids (Sum.inr j)
  | Sum.inr i =>
    (List.ofFn fun (j : Fin highCount) => bids (Sum.inl j)) ++
    (List.ofFn fun (j : Fin lowCount) => bids (Sum.inr j)).eraseldx ↑i
```

Direct retained dependencies

- [AppliedModelingLib.Auction.TwoValueAgent](#)

AppliedModelingLib.Auction.restrictBidsBySide

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → (Agent → Bool) → Bool → (Agent → ℝ) → Agent → ℝ

value:
fun {Agent : Type u_1} (side : Agent → Bool) (keep : Bool) (bids : Agent → ℝ) (a : Agent) =>
  if side a = keep then bids a else 0
```

AppliedModelingLib.Auction.saleCount

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → ℝ) → ℝ → ℕ

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → ℝ) (p : ℝ) => {i : Agent | p ≤ values i}.card
```

AppliedModelingLib.Auction.singlePriceRevenue

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → ℝ) → ℝ → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → ℝ) (p : ℝ) => ∑ i : Agent, if p ≤ values i then p else 0
```

AppliedModelingLib.Auction.candidateFixedPriceRevenue

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → ℝ) → ℕ → Agent → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → ℝ) (minWinners : ℕ) (i : Agent) =>
  if 0 ≤ values i ∧ minWinners ≤ AppliedModelingLib.Auction.saleCount values (values i) then
    AppliedModelingLib.Auction.singlePriceRevenue values (values i)
  else 0
```

Direct retained dependencies

- [AppliedModelingLib.Auction.saleCount](#)
- [AppliedModelingLib.Auction.singlePriceRevenue](#)

AppliedModelingLib.Auction.finiteCandidateFixedPriceBenchmark

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → ℕ → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (values : Agent → ℝ) (minWinners : ℕ) =>
  Finset.univ.sup' AppliedModelingLib.Auction.finiteCandidateFixedPriceBenchmark._proof_1
    (AppliedModelingLib.Auction.candidateFixedPriceRevenue values minWinners)
```

Direct retained dependencies

- [AppliedModelingLib.Auction.candidateFixedPriceRevenue](#)

AppliedModelingLib.Auction.finiteCandidateBenchmarkBidder

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → ℕ → Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (values : Agent → ℝ) (minWinners : ℕ) =>
  Classical.choose
    (AppliedModelingLib.Auction.exists_candidateFixedPriceRevenue_eq_finiteCandidateFixedPriceBenchmark values
      minWinners)
```

Direct retained dependencies

- [AppliedModelingLib.Auction.candidateFixedPriceRevenue](#)
- [AppliedModelingLib.Auction.finiteCandidateFixedPriceBenchmark](#)

AppliedModelingLib.Auction.finiteCandidateOfferPrice

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → ℕ → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (values : Agent → ℝ) (minWinners : ℕ) =>
  have i : Agent := AppliedModelingLib.Auction.finiteCandidateBenchmarkBidder values minWinners;
  if 0 ≤ values i ∧ minWinners ≤ AppliedModelingLib.Auction.saleCount values (values i) then values i else 0
```

Direct retained dependencies

- [AppliedModelingLib.Auction.finiteCandidateBenchmarkBidder](#)
- [AppliedModelingLib.Auction.saleCount](#)

AppliedModelingLib.Auction.thresholdPriceAuction

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} →
  [DecidableEq Agent] → ((Agent → ℝ) → Agent → ℝ) → AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [DecidableEq Agent] (threshold : (Agent → ℝ) → Agent → ℝ) =>
  { allocation := fun (bids : Agent → ℝ) (i : Agent) => if threshold bids i ≤ bids i then 1 else 0,
    payment := fun (bids : Agent → ℝ) (i : Agent) => if threshold bids i ≤ bids i then threshold bids i else 0 }
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)

AppliedModelingLib.Auction.totalBidValue

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → ℝ) → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → ℝ) => ∑ i : Agent, values i
```

AppliedModelingLib.Auction.twoValueBidProfile

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → (highCount lowCount : ℕ) → AppliedModelingLib.Auction.TwoValueAgent highCount lowCount → ℝ

value:
fun (H highCount lowCount : ℕ) (a : AppliedModelingLib.Auction.TwoValueAgent highCount lowCount) =>
  match a with
  | Sum.inl val => ↑H
  | Sum.inr val => 1
```

Direct retained dependencies

- [AppliedModelingLib.Auction.TwoValueAgent](#)

AppliedModelingLib.Auction.twoValueErasedBidList

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℕ → ℕ → List ℝ

value:
fun (H highCount lowCount : ℕ) => List.replicate highCount ↑H ++ List.replicate lowCount 1
```

AppliedModelingLib.Auction.twoValueFixedPriceBenchmark

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℕ → ℕ → ℝ

value:
fun (H highCount lowCount : ℕ) =>
  if hzero : highCount + lowCount = 0 then 0
  else
    AppliedModelingLib.Auction.finiteCandidateFixedPriceBenchmark
      (AppliedModelingLib.Auction.twoValueBidProfile H highCount lowCount) 1
```

Direct retained dependencies

- [AppliedModelingLib.Auction.TwoValueAgent](#)
- [AppliedModelingLib.Auction.finiteCandidateFixedPriceBenchmark](#)
- [AppliedModelingLib.Auction.twoValueBidProfile](#)

AppliedModelingLib.Auction.weightedPairingExpectedPayment

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [DecidableEq Agent] → (Agent → ℝ) → Agent → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (values : Agent → ℝ) (i : Agent) =>
  ∑ j : Agent,
    if j ≠ i ∧ values j ≤ values i then values j ^ 2 / (AppliedModelingLib.Auction.totalBidValue values - values i)
    else 0
```

Direct retained dependencies

- [AppliedModelingLib.Auction.totalBidValue](#)

AppliedModelingLib.Auction.weightedPairingExpectedRevenue

Declaration location: reusable library

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [DecidableEq Agent] → (Agent → ℝ) → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (values : Agent → ℝ) =>
  ∑ i : Agent, AppliedModelingLib.Auction.weightedPairingExpectedPayment values i
```

Direct retained dependencies

- [AppliedModelingLib.Auction.weightedPairingExpectedPayment](#)

AppliedModelingLib.FairCoin.fairMeasure

Declaration location: reusable library

Lean declaration body

```
type:
MeasureTheory.Measure Bool

value:
(PMF.bernoulli (1 / 2) AppliedModelingLib.FairCoin.fairMeasure._proof_2).toMeasure
```

AppliedModelingLib.FairCoin.productMeasure

Declaration location: reusable library

Lean declaration body

```
type:
(ι : Type u_1) → MeasureTheory.Measure (ι → Bool)

value:
fun (ι : Type u_1) => MeasureTheory.Measure.infinitePi fun (x : ι) => AppliedModelingLib.FairCoin.fairMeasure
```

Direct retained dependencies

- [AppliedModelingLib.FairCoin.fairMeasure](#)

AppliedModelingLib.pmfExp

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → [Fintype α] → PMF α → (α → ℝ) → ℝ

value:
fun {α : Type u_1} [Fintype α] (μ : PMF α) (f : α → ℝ) => ∑ a : α, (μ a).toReal * f a
```

AppliedModelingLib.pmfPi

Declaration location: reusable library

Lean declaration body

```
type:
{ι : Type u_1} →
[Fintype ι] →
[DecidableEq ι] → {α : ι → Type u_2} → [(i : ι) → Fintype (α i)] → ((i : ι) → PMF (α i)) → PMF ((i : ι) → α i)

value:
fun {ι : Type u_1} [Fintype ι] [DecidableEq ι] {α : ι → Type u_2} [(i : ι) → Fintype (α i)] (μ : (i : ι) → PMF (α i)) =>
  PMF.ofFintype (fun (f : (i : ι) → α i) => ∏ i : ι, (μ i) (f i)) (AppliedModelingLib.pmfPi._proof_1 μ)
```

AppliedModelingLib.pmfProb

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → [Fintype α] → PMF α → (p : α → Prop) → [DecidablePred p] → ℝ

value:
fun {α : Type u_1} [Fintype α] (μ : PMF α) (p : α → Prop) [DecidablePred p] =>
  AppliedModelingLib.pmfExp μ fun (a : α) => if p a then 1 else 0
```

Direct retained dependencies

- [AppliedModelingLib.pmfExp](#)

AppliedModelingLib.uniformPMF

Declaration location: reusable library

Lean declaration body

```
type:
(α : Type u_1) → [Fintype α] → [Nonempty α] → PMF α

value:
fun (α : Type u_1) [Fintype α] [Nonempty α] =>
  PMF.ofFintype (fun (x : α) => (↑ (Fintype.card α))-1) (AppliedModelingLib.uniformPMF._proof_1 α)
```

GHW01DigitalGoods.FixedSizeSampleSpace

Declaration location: paper-local

Lean declaration body

```
type:
(Agent : Type u_1) → [Fintype Agent] → [DecidableEq Agent] → ℕ → Finset (Finset Agent)

value:
fun (Agent : Type u_1) [Fintype Agent] [DecidableEq Agent] (sampleSize : ℕ) =>
  Finset.powersetCard sampleSize Finset.univ
```

GHW01DigitalGoods.PaperBidIndependentBoundaryRule

Declaration location: paper-local

Lean declaration body

```
field price:
GHW01DigitalGoods.PaperBidIndependentBoundaryRule → Multiset ℝ → ℝ

field acceptsEquality:
GHW01DigitalGoods.PaperBidIndependentBoundaryRule → Multiset ℝ → Bool
```

GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (values : Agent → ℝ) =>
  AppliedModelingLib.Auction.finiteCandidateFixedPriceBenchmark values 1
```

Direct retained dependencies

- [AppliedModelingLib.Auction.finiteCandidateFixedPriceBenchmark](#)

GHW01DigitalGoods.ProofBridge.ghwTightFixedPriceBenchmarkValue

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow (s : \mathbb{N}) \rightarrow 2 \leq s \rightarrow \mathbb{R}$ 

value:
fun (k s :  $\mathbb{N}$ ) (hs_two :  $2 \leq s$ ) =>
  have hs_pos :  $0 < s$  := GHW01DigitalGoods.ProofBridge.ghwTightFixedPriceBenchmarkValue._proof_2 s hs_two;
  GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark (AppliedModelingLib.Auction.ghwTightValue k s)
```

Direct retained dependencies

- [AppliedModelingLib.Auction.GhwTightAgent](#)
- [AppliedModelingLib.Auction.GhwTightLowerAgent](#)
- [AppliedModelingLib.Auction.ghwTightValue](#)
- [GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark](#)

GHW01DigitalGoods.ProofBridge.revenue

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → AppliedModelingLib.Auction.DigitalGoodsAuction Agent → (Agent →  $\mathbb{R}$ ) →  $\mathbb{R}$ 

value:
fun {Agent : Type u_1} [Fintype Agent] (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent)
  (values : Agent →  $\mathbb{R}$ ) =>
   $\sum i : \text{Agent}, \text{if } M.\text{allocation values } i = 1 \text{ then } M.\text{payment values } i \text{ else } 0$ 
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)

GHW01DigitalGoods.ProofBridge.truthful

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → AppliedModelingLib.Auction.DigitalGoodsAuction Agent → Prop

value:
fun {Agent : Type u_1} [DecidableEq Agent] (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) =>
 $\forall (values : \text{Agent} \rightarrow \mathbb{R}) (i : \text{Agent}) (\text{report} : \mathbb{R}),$ 
   $M.\text{utility values } i (\text{Function.update values } i \text{ report}) \leq M.\text{utility values } i \text{ values}$ 
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.utility](#)

GHW01DigitalGoods.ProofBridge.weightedPairingRevenue

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [DecidableEq Agent] → (Agent →  $\mathbb{R}$ ) →  $\mathbb{R}$ 

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (values : Agent →  $\mathbb{R}$ ) =>
  AppliedModelingLib.Auction.weightedPairingExpectedRevenue values
```

Direct retained dependencies

- [AppliedModelingLib.Auction.weightedPairingExpectedRevenue](#)

GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition

Declaration location: paper-local

Lean declaration body

```
field sample:
{Agent : Type u_1} →
  [inst : Fintype Agent] →
    [inst_1 : DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent → Finset Agent

field sample_nonempty:
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : DecidableEq Agent]
  (self : GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent), self.sample.Nonempty

field market_nonempty:
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : DecidableEq Agent]
  (self : GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent), (Finset.univ \ self.sample).Nonempty
```

GHW01DigitalGoods.SourceDefinitions.HasFiniteExpectedProfits

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent → Prop

value:
fun {Agent : Type u_1} [DecidableEq Agent] (auction : GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent) =>
  ∀ (values : GHW01DigitalGoods.SourceDefinitions.BidValues Agent) (i : Agent)
    (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent),
    Summable fun (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent) =>
      ((auction bids) outcome).toReal * outcome.profit values i
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.profit](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.BidValues](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomizedAuction](#)

GHW01DigitalGoods.SourceDefinitions.SupplyCapacity

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
ℕ
```

GHW01DigitalGoods.SourceDefinitions.ThresholdOffer

Declaration location: paper-local

Lean declaration body

```
field price:
GHW01DigitalGoods.SourceDefinitions.ThresholdOffer → ℝ

field boundary:
GHW01DigitalGoods.SourceDefinitions.ThresholdOffer → GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary](#)

GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan

Declaration location: paper-local

Lean declaration body

```
field sampleLaw:
{Agent : Type u_1} → GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent → PMF (Finset Agent)

field threshold:
{Agent : Type u_1} →
  GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent →
  Multiset ℝ → GHW01DigitalGoods.SourceDefinitions.ThresholdOffer
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.ThresholdOffer](#)

GHW01DigitalGoods.SourceDefinitions.biddersOnSide

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → Bool) → Bool → Finset Agent

value:
fun {Agent : Type u_1} [Fintype Agent] (side : Agent → Bool) (keep : Bool) => {i : Agent | side i = keep}
```

GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → Bool) → Bool → ℕ

value:
fun {Agent : Type u_1} [Fintype Agent] (side : Agent → Bool) (keep : Bool) =>
  (GHW01DigitalGoods.SourceDefinitions.biddersOnSide side keep).card
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.biddersOnSide](#)

GHW01DigitalGoods.SourceDefinitions.boundedSampleCapacityFloor

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
  [Fintype Agent] →
  (Agent → Bool) →
  Bool → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity

value:
fun {Agent : Type u_1} [Fintype Agent] (side : Agent → Bool) (sampleSide : Bool)
  (capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>
  GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side sampleSide * capacity /
  GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side !sampleSide
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide](#)

GHW01DigitalGoods.SourceDefinitions.certifiedFixedSizePartition

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[inst_1 : DecidableEq Agent] →
(sampleSize : ℕ) →
0 < sampleSize →
sampleSize < Fintype.card Agent →
1 (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize) →
GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (sampleSize : ℕ) (hsample_pos : 0 < sampleSize)
(hsample_lt : sampleSize < Fintype.card Agent)
(sample : 1 (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize)) =>
{ sample := ↑sample,
sample_nonempty := GHW01DigitalGoods.SourceDefinitions.exactHalfSample_nonempty hsample_pos sample,
market_nonempty := GHW01DigitalGoods.SourceDefinitions.fixedSizeMarket_nonempty hsample_lt sample }
```

Direct retained dependencies

- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition](#)

GHW01DigitalGoods.SourceDefinitions.optimalThresholdCandidates

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] → [Nonempty Agent] → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → Finset ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
{ p ∈ Finset.image bids Finset.univ |
GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue bids p =
GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue bids }
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue](#)
- [GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue](#)

GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceOutcome

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[Nonempty Agent] →
GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome bids (GHW01DigitalGoods.SourceDefinitions.optimalThreshold bids)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.optimalThreshold](#)

GHW01DigitalGoods.SourceDefinitions.optimalThresholdOnFinset

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → (sample : Finset Agent) → sample.Nonempty → (Agent → ℝ) → ℝ

value:
fun {Agent : Type u_1} [DecidableEq Agent] (sample : Finset Agent) (hnonempty : sample.Nonempty) (bids : Agent → ℝ) =>
  GHW01DigitalGoods.SourceDefinitions.optimalThreshold fun (i : 1 sample) => bids ↑i
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.optimalThreshold](#)

GHW01DigitalGoods.SourceDefinitions.directionalOptimalThresholdAuction

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[Nonempty Agent] →
[DecidableEq Agent] →
(sample : Finset Agent) →
sample.Nonempty →
GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary → AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [DecidableEq Agent] (sample : Finset Agent)
(hsample : sample.Nonempty) (boundary : GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary) =>
{
  allocation := fun (bids : Agent → ℝ) (i : Agent) =>
    have price : ℝ := GHW01DigitalGoods.SourceDefinitions.optimalThresholdOnFinset sample hsample bids;
    if i ∈ sample then 0
    else
      if GHW01DigitalGoods.SourceDefinitions.accepts { price := price, boundary := boundary } (bids i) then 1 else 0,
  payment := fun (bids : Agent → ℝ) (i : Agent) =>
    have price : ℝ := GHW01DigitalGoods.SourceDefinitions.optimalThresholdOnFinset sample hsample bids;
    if i ∈ sample then 0
    else
      if GHW01DigitalGoods.SourceDefinitions.accepts { price := price, boundary := boundary } (bids i) then price
      else 0 }
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdOffer](#)
- [GHW01DigitalGoods.SourceDefinitions.accepts](#)
- [GHW01DigitalGoods.SourceDefinitions.optimalThresholdOnFinset](#)

GHW01DigitalGoods.SourceDefinitions.otherBids

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] → [DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → Agent → Multiset ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent)
(i : Agent) =>
  Multiset.map bids (Finset.univ.erase i).val
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

GHW01DigitalGoods.SourceDefinitions.randomizedExpectedProfit

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent →
  GHW01DigitalGoods.SourceDefinitions.BidValues Agent →
  Agent → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → ℝ

value:
fun {Agent : Type u_1} [DecidableEq Agent] (auction : GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent)
  (values : GHW01DigitalGoods.SourceDefinitions.BidValues Agent) (i : Agent)
  (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
  Σ' (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent),
  ((auction bids) outcome).toReal * outcome.profit values i
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.profit](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.BidValues](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomizedAuction](#)

GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] →
  AppliedModelingLib.Auction.DigitalGoodsAuction Agent →
  (Agent → ℝ) → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent]
  (auction : AppliedModelingLib.Auction.DigitalGoodsAuction Agent) (bids : Agent → ℝ) =>
  { winners := {i : Agent | auction.allocation bids i = 1}, salePrice := auction.payment bids }
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)

GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → Finset Agent → (Agent → ℝ) → Multiset ℝ

value:
fun {Agent : Type u_1} (sample : Finset Agent) (bids : Agent → ℝ) => Multiset.map bids sample.val
```

GHW01DigitalGoods.SourceDefinitions.dualPriceRandomSamplingAuction

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] →
  Finset Agent →
  (Multiset ℝ → GHW01DigitalGoods.SourceDefinitions.ThresholdOffer) →
  AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (sample : Finset Agent)
  (f : Multiset ℝ → GHW01DigitalGoods.SourceDefinitions.ThresholdOffer) =>
```



```

{
allocation := fun (bids : Agent → ℝ) (i : Agent) =>
  have offer : GHW01DigitalGoods.SourceDefinitions.ThresholdOffer :=
    if i ∈ sample then f (GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset (Finset.univ \ sample) bids)
    else f (GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset sample bids);
  if GHW01DigitalGoods.SourceDefinitions.accepts offer (bids i) then 1 else 0,
payment := fun (bids : Agent → ℝ) (i : Agent) =>
  have offer : GHW01DigitalGoods.SourceDefinitions.ThresholdOffer :=
    if i ∈ sample then f (GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset (Finset.univ \ sample) bids)
    else f (GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset sample bids);
  if GHW01DigitalGoods.SourceDefinitions.accepts offer (bids i) then offer.price else 0 }

```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdOffer](#)
- [GHW01DigitalGoods.SourceDefinitions.accepts](#)
- [GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset](#)

GHW01DigitalGoods.SourceDefinitions.randomSamplingAuction

Declaration location: paper-local

Lean declaration body

```

type:
{Agent : Type u_1} →
[DecidableEq Agent] →
Finset Agent →
(Multiset ℝ → GHW01DigitalGoods.SourceDefinitions.ThresholdOffer) →
AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [DecidableEq Agent] (sample : Finset Agent)
(f : Multiset ℝ → GHW01DigitalGoods.SourceDefinitions.ThresholdOffer) =>
{
allocation := fun (bids : Agent → ℝ) (i : Agent) =>
  if i ∈ sample then 0
  else
    if
      GHW01DigitalGoods.SourceDefinitions.accepts
        (f (GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset sample bids)) (bids i) then
        1
      else 0,
payment := fun (bids : Agent → ℝ) (i : Agent) =>
  if i ∈ sample then 0
  else
    if
      GHW01DigitalGoods.SourceDefinitions.accepts
        (f (GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset sample bids)) (bids i) then
        (f (GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset sample bids)).price
      else 0 }

```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdOffer](#)
- [GHW01DigitalGoods.SourceDefinitions.accepts](#)
- [GHW01DigitalGoods.SourceDefinitions.sampleBidMultiset](#)

GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcome

Declaration location: paper-local

Lean declaration body

```

type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] → (Agent → ℝ) → (Agent → Agent) → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ) (offer : Agent → Agent) =>
{ winners := {i : Agent | bids (offer i) ≤ bids i}, salePrice := fun (i : Agent) => bids (offer i) }

```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)

GHW01DigitalGoods.Theorem93.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel

Declaration location: paper-local

Lean declaration body

```
field auctionFamily:
{ _highValue : ℕ} →
  GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel _highValue →
    (highCount lowCount : ℕ) →
      AppliedModelingLib.Auction.DigitalGoodsAuction (AppliedModelingLib.Auction.TwoValueAgent highCount lowCount)

field truthful:
∀ { _highValue : ℕ} (self : GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel _highValue)
  (highCount lowCount : ℕ),
    AppliedModelingLib.Auction.paper_digital_goods_truthful (self.auctionFamily highCount lowCount)

field binary:
∀ { _highValue : ℕ} (self : GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel _highValue)
  (highCount lowCount : ℕ), (self.auctionFamily highCount lowCount).BinaryAllocation

field individuallyRational:
∀ { _highValue : ℕ} (self : GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel _highValue)
  (highCount lowCount : ℕ), (self.auctionFamily highCount lowCount).IndividuallyRational

field noPositiveTransfers:
∀ { _highValue : ℕ} (self : GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel _highValue)
  (highCount lowCount : ℕ), (self.auctionFamily highCount lowCount).NoPositiveTransfers

field focusedOutcome:
{ _highValue : ℕ} →
  GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel _highValue → Multiset ℝ → ℝ → Option ℝ

field focusedOutcome_represents:
∀ { _highValue : ℕ} (self : GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel _highValue)
  {highCount lowCount : ℕ} (bids : AppliedModelingLib.Auction.TwoValueAgent highCount lowCount → ℝ)
  (i : AppliedModelingLib.Auction.TwoValueAgent highCount lowCount),
    AppliedModelingLib.Auction.deterministicAuctionOffer (self.auctionFamily highCount lowCount) bids i =
      self.focusedOutcome ↑ (AppliedModelingLib.Auction.paper_theorem9_3_erased_report_list bids i)
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.BinaryAllocation](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.IndividuallyRational](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.NoPositiveTransfers](#)
- [AppliedModelingLib.Auction.TwoValueAgent](#)
- [AppliedModelingLib.Auction.deterministicAuctionOffer](#)
- [AppliedModelingLib.Auction.paper_digital_goods_truthful](#)
- [AppliedModelingLib.Auction.paper_theorem9_3_erased_report_list](#)

GHW01DigitalGoods.WeightedPairingSourceProfile

Declaration location: paper-local

Lean declaration body

```
field bid_nonneg:
∀ {Agent : Type u_1} [inst : Fintype Agent] {bids : Agent → ℝ},
  GHW01DigitalGoods.WeightedPairingSourceProfile bids → ∀ (i : Agent), 0 ≤ bids i

field positive_other_bid_mass:
∀ {Agent : Type u_1} [inst : Fintype Agent] {bids : Agent → ℝ},
  GHW01DigitalGoods.WeightedPairingSourceProfile bids →
    ∀ (i : Agent), 0 < AppliedModelingLib.Auction.totalBidValue bids - bids i
```

Direct retained dependencies

- [AppliedModelingLib.Auction.totalBidValue](#)

GHW01DigitalGoods.assumption_theorem9_3_primitive_set_of_bids_deterministic_source_model

Declaration location: paper-local

Lean declaration body

type:
 $\mathbb{N} \rightarrow \text{Type}$

value:
fun (highValue : $\mathbb{N}$) => GHW01DigitalGoods.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel highValue

Direct retained dependencies

- [GHW01DigitalGoods.Theorem93.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel](#)

GHW01DigitalGoods.boundedEligible

Declaration location: paper-local

Lean declaration body

type:
{Agent : Type u_1} → (Agent → Bool) → Bool → (Agent → $\mathbb{R}$) → $\mathbb{R}$ → Agent → Prop

value:
fun {Agent : Type u_1} (side : Agent → Bool) (marketSide : Bool) (bids : Agent → $\mathbb{R}$) (price : $\mathbb{R}$) (i : Agent) =>
side i = marketSide $\wedge$ price $\leq$ bids i

GHW01DigitalGoods.boundedSupplySinglePriceRevenue

Declaration location: paper-local

Lean declaration body

type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → $\mathbb{R}$) → $\mathbb{N}$ → $\mathbb{R}$ → $\mathbb{R}$

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → $\mathbb{R}$) (capacity : $\mathbb{N}$) (p : $\mathbb{R}$) =>
min $\uparrow$ capacity $\uparrow$ (AppliedModelingLib.Auction.saleCount values p) * p

Direct retained dependencies

- [AppliedModelingLib.Auction.saleCount](#)

GHW01DigitalGoods.SourceDefinitions.boundedCandidateRevenueAtPrice

Declaration location: paper-local

Lean declaration body

type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → $\mathbb{R}$) → $\mathbb{N}$ → $\mathbb{R}$ → $\mathbb{R}$

value:
fun {Agent : Type u_1} [Fintype Agent] (bids : Agent → $\mathbb{R}$) (capacity : $\mathbb{N}$) (price : $\mathbb{R}$) =>
if 0 $\leq$ price then GHW01DigitalGoods.boundedSupplySinglePriceRevenue bids capacity price else 0

Direct retained dependencies

- [GHW01DigitalGoods.boundedSupplySinglePriceRevenue](#)

GHW01DigitalGoods.candidateBoundedSupplyRevenue

Declaration location: paper-local

Lean declaration body

type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → $\mathbb{R}$) → $\mathbb{N}$ → Agent → $\mathbb{R}$

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → $\mathbb{R}$) (capacity : $\mathbb{N}$) (i : Agent) =>
if 0 $\leq$ values i then GHW01DigitalGoods.boundedSupplySinglePriceRevenue values capacity (values i) else 0

Direct retained dependencies

- [GHW01DigitalGoods.boundedSupplySinglePriceRevenue](#)

GHW01DigitalGoods.boundedSupplyFixedPriceBenchmark

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → ℕ → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (values : Agent → ℝ) (capacity : ℕ) =>
  Finset.univ.sup' GHW01DigitalGoods.boundedSupplyFixedPriceBenchmark._proof_1
    (GHW01DigitalGoods.candidateBoundedSupplyRevenue values capacity)
```

Direct retained dependencies

- [GHW01DigitalGoods.candidateBoundedSupplyRevenue](#)

GHW01DigitalGoods.SourceDefinitions.boundedOptimalPriceMaximizers

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → ℕ → Finset ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : Agent → ℝ) (capacity : ℕ) =>
  {price ∈ AppliedModelingLib.Auction.finiteCandidatePriceSet bids |
    GHW01DigitalGoods.SourceDefinitions.boundedCandidateRevenueAtPrice bids capacity price =
    GHW01DigitalGoods.boundedSupplyFixedPriceBenchmark bids capacity}
```

Direct retained dependencies

- [AppliedModelingLib.Auction.finiteCandidatePriceSet](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedCandidateRevenueAtPrice](#)
- [GHW01DigitalGoods.boundedSupplyFixedPriceBenchmark](#)

GHW01DigitalGoods.SourceDefinitions.canonicalBoundedOptimalThreshold

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → ℕ → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : Agent → ℝ) (capacity : ℕ) =>
  (GHW01DigitalGoods.SourceDefinitions.boundedOptimalPriceMaximizers bids capacity).max'
    (GHW01DigitalGoods.SourceDefinitions.boundedOptimalPriceMaximizers_nonempty bids capacity)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.boundedOptimalPriceMaximizers](#)

GHW01DigitalGoods.SourceDefinitions.boundedOptimalThresholdOnFinset

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → (sample : Finset Agent) → sample.Nonempty → (Agent → ℝ) → ℕ → ℝ

value:
fun {Agent : Type u_1} [DecidableEq Agent] (sample : Finset Agent) (hnonempty : sample.Nonempty) (bids : Agent → ℝ)
  (capacity : ℕ) =>
  GHW01DigitalGoods.SourceDefinitions.canonicalBoundedOptimalThreshold (fun (i : ↑sample) => bids ↑i) capacity
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.canonicalBoundedOptimalThreshold](#)

GHW01DigitalGoods.SourceDefinitions.canonicalBoundedSamplingPriceRule

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[Nonempty Agent] →
[DecidableEq Agent] →
(side : Agent → Bool) →
(sampleSide : Bool) →
ℕ → 0 < GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side sampleSide → (Agent → ℝ) → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [DecidableEq Agent] (side : Agent → Bool) (sampleSide : Bool)
(sampleCapacity : ℕ) (hsample_pos : 0 < GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side sampleSide)
(bids : Agent → ℝ) =>
GHW01DigitalGoods.SourceDefinitions.boundedOptimalThresholdOnFinset
(GHW01DigitalGoods.SourceDefinitions.biddersOnSide side sampleSide)
(GHW01DigitalGoods.SourceDefinitions.canonicalBoundedSamplingPriceRule._proof_1 side sampleSide hsample_pos) bids
sampleCapacity
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide](#)
- [GHW01DigitalGoods.SourceDefinitions.biddersOnSide](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedOptimalThresholdOnFinset](#)

GHW01DigitalGoods.SourceDefinitions.sellsFullCapacityAtOptimalPrice

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → Prop

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : Agent → ℝ)
(capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>
∃ (price : ℝ),
GHW01DigitalGoods.SourceDefinitions.IsBoundedSupplyOptimalThreshold bids capacity price ∧
capacity ≤ AppliedModelingLib.Auction.saleCount bids price
```

Direct retained dependencies

- [AppliedModelingLib.Auction.saleCount](#)
- [GHW01DigitalGoods.SourceDefinitions.IsBoundedSupplyOptimalThreshold](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)

GHW01DigitalGoods.earlierBoundedEligibleCount

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [LinearOrder Agent] → (Agent → Bool) → Bool → (Agent → ℝ) → ℝ → Agent → ℕ

value:
fun {Agent : Type u_1} [Fintype Agent] [LinearOrder Agent] (side : Agent → Bool) (marketSide : Bool) (bids : Agent → ℝ)
(price : ℝ) (i : Agent) =>
{j : Agent | GHW01DigitalGoods.boundedEligible side marketSide bids price j ∧ j < i}.card
```

Direct retained dependencies

- [GHW01DigitalGoods.boundedEligible](#)

GHW01DigitalGoods.boundedCappedWinner

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [LinearOrder Agent] → (Agent → Bool) → Bool → (Agent → ℝ) → ℝ → ℕ → Agent → Prop

value:
fun {Agent : Type u_1} [Fintype Agent] [LinearOrder Agent] (side : Agent → Bool) (marketSide : Bool) (bids : Agent → ℝ)
  (price : ℝ) (capacity : ℕ) (i : Agent) =>
  GHW01DigitalGoods.boundedEligible side marketSide bids price i ∧
  GHW01DigitalGoods.earlierBoundedEligibleCount side marketSide bids price i < capacity
```

Direct retained dependencies

- [GHW01DigitalGoods.boundedEligible](#)
- [GHW01DigitalGoods.earlierBoundedEligibleCount](#)

GHW01DigitalGoods.boundedCappedThresholdAuction

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[LinearOrder Agent] →
(Agent → Bool) → Bool → ((Agent → ℝ) → ℝ) → ℕ → AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [LinearOrder Agent] (side : Agent → Bool) (marketSide : Bool)
  (priceRule : (Agent → ℝ) → ℝ) (capacity : ℕ) =>
{
  allocation := fun (bids : Agent → ℝ) (i : Agent) =>
    if GHW01DigitalGoods.boundedCappedWinner side marketSide bids (priceRule bids) capacity i then 1 else 0,
  payment := fun (bids : Agent → ℝ) (i : Agent) =>
    if GHW01DigitalGoods.boundedCappedWinner side marketSide bids (priceRule bids) capacity i then priceRule bids
    else 0 }

```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.boundedCappedWinner](#)

GHW01DigitalGoods.fixedSizeHitCount

Declaration location: paper-local

Lean declaration body

```
type:
{α : Type u_1} → [DecidableEq α] → Finset α → Finset α → ℕ

value:
fun {α : Type u_1} [DecidableEq α] (B sample : Finset α) => (sample n B).card
```

GHW01DigitalGoods.fixedSizeSampleProbability

Declaration location: paper-local

Lean declaration body

```
type:
{α : Type u_1} → [DecidableEq α] → Finset α → ℕ → (event : Finset α → Prop) → [DecidablePred event] → ℝ

value:
fun {α : Type u_1} [DecidableEq α] (A : Finset α) (k : ℕ) (event : Finset α → Prop) [DecidablePred event] =>
  ↑(Finset.filter event (Finset.powersetCard k A)).card / ↑(Finset.powersetCard k A).card
```

GHW01DigitalGoods.paperBidIndependentBoundaryPayment

Declaration location: paper-local

Lean declaration body

```
type:
GHW01DigitalGoods.PaperBidIndependentBoundaryRule → Multiset ℝ → ℝ → ℝ

value:
fun (rule : GHW01DigitalGoods.PaperBidIndependentBoundaryRule) (others : Multiset ℝ) (bid : ℝ) =>
  if rule.acceptsEquality others = true then if rule.price others ≤ bid then rule.price others else 0
  else if rule.price others < bid then rule.price others else 0
```

Direct retained dependencies

- [GHW01DigitalGoods.PaperBidIndependentBoundaryRule](#)

GHW01DigitalGoods.paper_theorem8_2_continuous_marginal_expected_revenue

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → (Agent → ℝ) → (Agent → MeasureTheory.Measure ℝ) → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] (values : Agent → ℝ) (offerLaw : Agent → MeasureTheory.Measure ℝ) =>
  ∑ i : Agent, ∫ (p : ℝ), if p ≤ values i then p else 0 ∂offerLaw i
```

GHW01DigitalGoods.pmfEventProbability

Declaration location: paper-local

Lean declaration body

```
type:
{Ω : Type u_1} → [Fintype Ω] → [DecidableEq Ω] → PMF Ω → (Ω → Prop) → ℝ

value:
fun {Ω : Type u_1} [Fintype Ω] [DecidableEq Ω] (law : PMF Ω) (P : Ω → Prop) => AppliedModelingLib.pmfProb law P
```

Direct retained dependencies

- [AppliedModelingLib.pmfProb](#)

GHW01DigitalGoods.randomSamplingOptimalThresholdAuction

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
  [Fintype Agent] →
  [Nonempty Agent] →
  [DecidableEq Agent] → (Agent → Bool) → Bool → ℕ → AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [DecidableEq Agent] (side : Agent → Bool) (sampleSide : Bool)
  (minWinners : ℕ) =>
  {
    allocation := fun (bids : Agent → ℝ) (i : Agent) =>
      if side i = sampleSide then 0
      else
        if
          AppliedModelingLib.Auction.finiteCandidateOfferPrice
            (AppliedModelingLib.Auction.restrictBidsBySide side sampleSide bids) minWinners ≤
              bids i then
            1
          else 0,
    payment := fun (bids : Agent → ℝ) (i : Agent) =>
      if side i = sampleSide then 0
      else
        if
          AppliedModelingLib.Auction.finiteCandidateOfferPrice
            (AppliedModelingLib.Auction.restrictBidsBySide side sampleSide bids) minWinners ≤
              bids i then
            AppliedModelingLib.Auction.finiteCandidateOfferPrice
              (AppliedModelingLib.Auction.restrictBidsBySide side sampleSide bids) minWinners
          else 0 }
  }
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.finiteCandidateOfferPrice](#)
- [AppliedModelingLib.Auction.restrictBidsBySide](#)

GHW01DigitalGoods.sampledSideAssignment

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → Finset Agent → Agent → Bool

value:
fun {Agent : Type u_1} [DecidableEq Agent] (sample : Finset Agent) (i : Agent) => decide (i ∈ sample)
```

GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition.side

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[inst_1 : DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent → Agent → Bool

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent]
(partition : GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent) (a : Agent) =>
GHW01DigitalGoods.sampledSideAssignment partition.sample a
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition](#)
- [GHW01DigitalGoods.sampledSideAssignment](#)

GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceCertifiedOutcomeLaw

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[Nonempty Agent] →
[inst_2 : LinearOrder Agent] →
PMF (GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent) →
GHW01DigitalGoods.SourceDefinitions.SupplyCapacity →
(Agent → ℝ) → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [LinearOrder Agent]
(partitionLaw : PMF (GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent))
(capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) (bids : Agent → ℝ) =>
PMF.map
(fun (partition : GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition Agent) =>
GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome
(GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceSampling partition.side true capacity
(GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceCertifiedOutcomeLaw._proof_1 partition)
(GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceCertifiedOutcomeLaw._proof_2 partition))
bids)
partitionLaw
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition](#)
- [GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition.side](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceSampling](#)
- [GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome](#)

GHW01DigitalGoods.SourceDefinitions.canonicalBoundedDualPriceSamplingAuction

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[Nonempty Agent] →
[inst_2 : LinearOrder Agent] →
(sample : Finset Agent) →
sample.Nonempty →
(Finset.univ \ sample).Nonempty →
GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [LinearOrder Agent] (sample : Finset Agent)
(hsample : sample.Nonempty) (hmarket : (Finset.univ \ sample).Nonempty)
(halfCapacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>
have side : Agent → Bool := GHW01DigitalGoods.sampledSideAssignment sample;
have left : AppliedModelingLib.Auction.DigitalGoodsAuction Agent :=
GHW01DigitalGoods.boundedCappedThresholdAuction side false
(fun (bids : Agent → ℝ) =>
GHW01DigitalGoods.SourceDefinitions.boundedOptimalThresholdOnFinset sample hsample bids halfCapacity)
halfCapacity;
have right : AppliedModelingLib.Auction.DigitalGoodsAuction Agent :=
GHW01DigitalGoods.boundedCappedThresholdAuction side true
(fun (bids : Agent → ℝ) =>
GHW01DigitalGoods.SourceDefinitions.boundedOptimalThresholdOnFinset (Finset.univ \ sample) hmarket bids
halfCapacity)
halfCapacity;
{
allocation := fun (bids : Agent → ℝ) (i : Agent) =>
if side i = false then left.allocation bids i else right.allocation bids i,
payment := fun (bids : Agent → ℝ) (i : Agent) =>
if side i = false then left.payment bids i else right.payment bids i }
}
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedOptimalThresholdOnFinset](#)
- [GHW01DigitalGoods.boundedCappedThresholdAuction](#)
- [GHW01DigitalGoods.sampledSideAssignment](#)

GHW01DigitalGoods.twoValueBidIndependentBoundaryRevenue

Declaration location: paper-local

Lean declaration body

```
type:
GHW01DigitalGoods.PaperBidIndependentBoundaryRule → ℕ → ℕ → ℕ → ℝ

value:
fun (rule : GHW01DigitalGoods.PaperBidIndependentBoundaryRule) (highValue highCount lowCount : ℕ) =>
↑highCount *
GHW01DigitalGoods.paperBidIndependentBoundaryPayment rule
↑(AppliedModelingLib.Auction.twoValueErasedBidList highValue (highCount - 1) lowCount) ↑highValue +
↑lowCount *
GHW01DigitalGoods.paperBidIndependentBoundaryPayment rule
(↑(AppliedModelingLib.Auction.twoValueErasedBidList highValue highCount (lowCount - 1))) 1
```

Direct retained dependencies

- [AppliedModelingLib.Auction.twoValueErasedBidList](#)
- [GHW01DigitalGoods.PaperBidIndependentBoundaryRule](#)
- [GHW01DigitalGoods.paperBidIndependentBoundaryPayment](#)

GHW01DigitalGoods.uniformFixedSizeSampleLaw

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[inst_1 : DecidableEq Agent] →
(sampleSize : ℕ) →
sampleSize ≤ Fintype.card Agent → PMF 1 (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize)

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (sampleSize : ℕ) (hsize : sampleSize ≤ Fintype.card Agent) =>
AppliedModelingLib.uniformPMF 1 (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize)
```

Direct retained dependencies

- [AppliedModelingLib.uniformPMF](#)
- [GHW01DigitalGoods.FixedSizeSampleSpace](#)

GHW01DigitalGoods.SourceDefinitions.exactHalfSampleLaw

Declaration location: paper-local

Lean declaration body

```
type:
(Agent : Type u_1) →
[inst : Fintype Agent] →
[inst_1 : DecidableEq Agent] →
(sampleSize : ℕ) →
0 < sampleSize →
Fintype.card Agent = sampleSize + sampleSize → PMF 1 (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize)

value:
fun (Agent : Type u_1) [Fintype Agent] [DecidableEq Agent] (sampleSize : ℕ) (_hsample_pos : 0 < sampleSize)
(hhalf : Fintype.card Agent = sampleSize + sampleSize) =>
GHW01DigitalGoods.uniformFixedSizeSampleLaw sampleSize
(GHW01DigitalGoods.SourceDefinitions.exactHalfSampleLaw._proof_2 Agent sampleSize hhalf)
```

Direct retained dependencies

- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.uniformFixedSizeSampleLaw](#)

GHW01DigitalGoods.weightedPairingOfferProbability

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} → [Fintype Agent] → [DecidableEq Agent] → (Agent → ℝ) → Agent → Agent → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ) (i j : Agent) =>
if j = i then 0 else bids j / (AppliedModelingLib.Auction.totalBidValue bids - bids i)
```

Direct retained dependencies

- [AppliedModelingLib.Auction.totalBidValue](#)

GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferLaw

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] → (bids : Agent → ℝ) → GHW01DigitalGoods.WeightedPairingSourceProfile bids → Agent → PMF Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
(source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) (i : Agent) =>
PMF.ofFintype (fun (j : Agent) => ENNReal.ofReal (GHW01DigitalGoods.weightedPairingOfferProbability bids i j))
(GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferLaw._proof_1 bids source i)
```

Direct retained dependencies

- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)
- [GHW01DigitalGoods.weightedPairingOfferProbability](#)

GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferFunctionLaw

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] → (bids : Agent → ℝ) → GHW01DigitalGoods.WeightedPairingSourceProfile bids → PMF (Agent → Agent)

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
  (source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) =>
  AppliedModelingLib.pmfPi fun (i : Agent) => GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferLaw bids source i
```

Direct retained dependencies

- [AppliedModelingLib.pmfPi](#)
- [GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferLaw](#)
- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)

GHW01DigitalGoods.weightedPairingSourceExpectedPayment

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] → (bids : Agent → ℝ) → GHW01DigitalGoods.WeightedPairingSourceProfile bids → Agent → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
  (source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) (i : Agent) =>
  ∑ j : Agent, GHW01DigitalGoods.weightedPairingOfferProbability bids i j * if bids j ≤ bids i then bids j else 0
```

Direct retained dependencies

- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)
- [GHW01DigitalGoods.weightedPairingOfferProbability](#)

GHW01DigitalGoods.weightedPairingSourceOperationalExpectedRevenue

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] → (bids : Agent → ℝ) → GHW01DigitalGoods.WeightedPairingSourceProfile bids → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
  (source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) =>
  ∑ i : Agent, GHW01DigitalGoods.weightedPairingSourceExpectedPayment bids source i
```

Direct retained dependencies

- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)
- [GHW01DigitalGoods.weightedPairingSourceExpectedPayment](#)

GHW01DigitalGoods.SourceDefinitions.weightedPairingAuctionRevenue

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] → (bids : Agent → ℝ) → GHW01DigitalGoods.WeightedPairingSourceProfile bids → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
  (source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) =>
  GHW01DigitalGoods.weightedPairingSourceOperationalExpectedRevenue bids source
```

Direct retained dependencies

- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)
- [GHW01DigitalGoods.weightedPairingSourceOperationalExpectedRevenue](#)

GHW01DigitalGoods.SourceDefinitions.weightedPairingW

Declaration location: paper-local

Lean declaration body

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] → (bids : Agent → ℝ) → GHW01DigitalGoods.WeightedPairingSourceProfile bids → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
  (source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) =>
  GHW01DigitalGoods.SourceDefinitions.weightedPairingAuctionRevenue bids source
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.weightedPairingAuctionRevenue](#)
- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)

Paper-specific semantic prerequisites

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome

Source locator: cited publication, lines 200-203

Verbatim paper-source connection

Given a set of bids, the outcome of an auction is the subset of bids that are satisfied and a corresponding set of sale prices such that, for each winning bid bi , the associated sale price is at most bi . A deterministic auction mechanism maps sets of bids to auction

Lean-elaborated paper signature

field winners:

{Agent : Type u_1} → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent → Finset Agent

field salePrice:

{Agent : Type u_1} → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent → Agent → $\mathbb{R}$

Recorded source-to-declaration screening Verdict: matches

Reason: The actual carrier records a finite winner subset and bidder-indexed sale prices. The separately displayed FillsAtOrBelow predicate supplies the source200-203 winner-price constraint; correctness and concrete outcome uses retain that constraint. Unused loser prices do not alter outcomes or revenue.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeLaw

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeLaw

Source locator: cited publication, lines 495-508

Verbatim paper-source connection

Let k_s be the number of bids satisfied by optimal single-price or dual-price. In this section we describe fixed pricing on the sample (i.e. the number of bids a multi-price auction. The weighted pairing auction in the sample that are at least $\text{opt}(B')$) and let k_n we present is in the bid-independent class with the be the number of bids in the non-sample that are at function f defined as follows:

least $\text{opt}(B')$. If $F' \geq F/3$ then $k_s \geq \alpha/3$.

$f(B) = b \in B$ w.p. $[U+0002 \text{ control character}]' b b'$

Now, assuming that $k_s \geq \alpha/3$, we show that the $b \in B$

probability that $k_n < k_s/2$ is small. Note that $k_n <$

$k_s/2$ implies that among the top $(3/2)k_s$ bids, at least Thus, to determine if bidder i wins the auction and

k_s are in the sample. Note that the sample and the at what price, pick a bid $b \in B_i$ with probability $\text{pr}_{\text{non-sample}}$ are symmetric: taking a random subset

$\hookrightarrow$ proportional to the value of b , i.e., $b/(T - b_i)$. This pairs

containing half of the elements in A is equivalent to b_i with b . If $b \leq b_i$, bidder i wins at cost b , othertaking a complement of a random subset of half of wise

$\hookrightarrow$ i loses. Note that the result of this selection for

the elements. We apply Lemma 6.1 with S being i does not affect the auction outcome for the bidder

the non-sample, B being the top $i = (3/2)k_s$ bids, who bid b .

Lean-expanded paper semantic target

type:

```
{Agent : Type u_1} →  
  [inst : Fintype Agent] →  
  [DecidableEq Agent] →  
  (bids : Agent → ℝ) →  
    GHW01DigitalGoods.WeightedPairingSourceProfile bids →  
    PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)
```

value:

```
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)  
  (source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) =>  
  PMF.map (GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcome bids)  
    (GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferFunctionLaw bids source)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferFunctionLaw](#)
- [GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcome](#)
- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual joint outcome PMF maps the independent bidderwise other-bid draws into winners and their offered prices. Its displayed profile supplies nonnegative bids and positive other-bid mass; the direct source wrapper derives those from the bound normalized/non-singleton domain. This is source495-508 under the bound joint-law convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.ProofBridge.source_weightedPairingAuction

Paper-local declaration:

GHW01DigitalGoods.ProofBridge.source_weightedPairingAuction

Source locator: cited publication, lines 495-508

Verbatim paper-source connection

Let k_s be the number of bids satisfied by optimal single-price or dual-price. In this section we describe fixed pricing on the sample (i.e. the number of bids a multi-price auction. The weighted pairing auction in the sample that are at least $\text{opt}(B')$) and let k_n we present is in the bid-independent class with the be the number of bids in the non-sample that are at function f defined as follows:

least $\text{opt}(B')$. If $F' \geq F/3$ then $k_s \geq \alpha/3$.

$f(B) = b \in B$ w.p. $[U+0002 \text{ control character}]' b b'$

Now, assuming that $k_s \geq \alpha/3$, we show that the $b \in B$

probability that $k_n < k_s/2$ is small. Note that $k_n <$

$k_s/2$ implies that among the top $(3/2)k_s$ bids, at least Thus, to determine if bidder i wins the auction and

k_s are in the sample. Note that the sample and the at what price, pick a bid $b \in B_i$ with probability prono-sample are symmetric: taking a random subset

$\hookrightarrow$ portional to the value of b , i.e., $b/(T - b_i)$. This pairs

containing half of the elements in A is equivalent to b_i with b . If $b \leq b_i$, bidder i wins at cost b , othertaking a complement of a random subset of half of wise

$\hookrightarrow$ i loses. Note that the result of this selection for

the elements. We apply Lemma 6.1 with S being i does not affect the auction outcome for the bidder

the non-sample, B being the top $i = (3/2)k_s$ bids, who bid b .

Lean-expanded paper semantic target

type:

{Agent : Type u_1} →

[Fintype Agent] →

(bids : Agent → ℝ) →

(∀ (i : Agent), 1 ≤ bids i) →

(∀ (i : Agent), ∃ (j : Agent), j ≠ i) → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

value:

fun {Agent : Type u_1} [Fintype Agent] (bids : Agent → ℝ) (hbid_ge_one : ∀ (i : Agent), 1 ≤ bids i)

(hdistinct : ∀ (i : Agent), ∃ (j : Agent), j ≠ i) =>

GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeLaw bids

(GHW01DigitalGoods.ProofBridge.source_weightedPairingAuction._proof_1 bids hbid_ge_one hdistinct)

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeLaw](#)
- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual auction is the joint outcome law obtained from independent bidderwise draws with probability $b_j/(T-b_i)$, zero self-draw probability, acceptance $b_j \leq b_i$ and payment b_j . Its visible unit lower bound and distinct-other-bidder premises supply the exact bound probability-domain and joint-law conventions for source495-508.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue

Source locator: cited publication, lines 211-214

Verbatim paper-source connection

outcomes. We use R to denote the auction revenue for a particular auction mechanism and set of bids. R is the sum of all sale prices. For randomized auctions, R is a random variable.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

type:
 $\{\text{Agent} : \text{Type } u_1\} \rightarrow \text{GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent} \rightarrow \mathbb{R}$

value:
 $\text{fun } \{\text{Agent} : \text{Type } u_1\} (\text{outcome} : \text{GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent}) \Rightarrow \sum i \in \text{outcome.winners, outcome.salePrice } i$

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual revenue is the finite sum of sale prices over winners only, exactly source211-214. It does not add loser payments.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeExpectedRevenue

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeExpectedRevenue

Source locator: cited publication, lines 621-623

Verbatim paper-source connection

We use the following notation. We denote the expected revenue of the $\sqrt{}$ wighted pairing auction by W ,
let $k = \log h$ and $s = \log h$ First we give bound the

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] → (bids : Agent → ℝ) → GHW01DigitalGoods.WeightedPairingSourceProfile bids → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
(source : GHW01DigitalGoods.WeightedPairingSourceProfile bids) =>
AppliedModelingLib.pmfExp (GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferFunctionLaw bids source)
fun (offer : Agent → Agent) => (GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcome bids offer).revenue
```

Direct retained dependencies

- [AppliedModelingLib.pmfExp](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue](#)
- [GHW01DigitalGoods.SourceDefinitions.weightedPairingOfferFunctionLaw](#)
- [GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcome](#)
- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual expected revenue averages winner-price revenue over the finite joint offer-function law. The displayed independent draws, zero self-probability and $b_j/(T-b_i)$ weights give source W under the bound probability-domain and joint-law conventions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.ProofBridge.source_weightedPairingOutcomeExpectedRevenue

Paper-local declaration:

GHW01DigitalGoods.ProofBridge.source_weightedPairingOutcomeExpectedRevenue

Source locator: cited publication, lines 621-623

Verbatim paper-source connection

We use the following notation. We denote the expected revenue of the $\sqrt{}$ wighted pairing auction by W ,
let $k = \log h$ and $s = \log h$ First we give bound the

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[FIntype Agent] → (bids : Agent → ℝ) → (∀ (i : Agent), 1 ≤ bids i) → (∀ (i : Agent), ∃ (j : Agent), j ≠ i) → ℝ

value:
fun {Agent : Type u_1} [FIntype Agent] (bids : Agent → ℝ) (hbid_ge_one : ∀ (i : Agent), 1 ≤ bids i)
(hdistinct : ∀ (i : Agent), ∃ (j : Agent), j ≠ i) =>
GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeExpectedRevenue bids
(GHW01DigitalGoods.ProofBridge.source_weightedPairingAuction._proof_1 bids hbid_ge_one hdistinct)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeExpectedRevenue](#)
- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual value is finite expected revenue of the operational weighted-pairing outcome law. The displayed payment, joint draw and revenue bodies sum winner prices under the source probabilities; visible normalized bids and another bidder supply the bound domain convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.ProofBridge.source_weightedPairingW

Paper-local declaration:

GHW01DigitalGoods.ProofBridge.source_weightedPairingW

Source locator: cited publication, lines 621-623

Verbatim paper-source connection

We use the following notation. We denote the expected revenue of the $\sqrt{}$ wighted pairing auction by W ,
let $k = \log h$ and $s = \log h$ First we give bound the

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[FIntype Agent] → (bids : Agent → ℝ) → (∀ (i : Agent), 1 ≤ bids i) → (∀ (i : Agent), ∃ (j : Agent), j ≠ i) → ℝ

value:
fun {Agent : Type u_1} [FIntype Agent] (bids : Agent → ℝ) (hbid_ge_one : ∀ (i : Agent), 1 ≤ bids i)
(hdistinct : ∀ (i : Agent), ∃ (j : Agent), j ≠ i) =>
GHW01DigitalGoods.SourceDefinitions.weightedPairingW bids
(GHW01DigitalGoods.ProofBridge.source_weightedPairingAuction._proof_1 bids hbid_ge_one hdistinct)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.weightedPairingW](#)
- [GHW01DigitalGoods.WeightedPairingSourceProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual W expands to the sum over bidders and other-bid draws of draw probability times the offered price when accepted. This is the source621-623 expected revenue, on the explicit normalized/non-singleton domain and under the bound joint-law convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.ProofBridge.totalValue

Paper-local declaration:

GHW01DigitalGoods.ProofBridge.totalValue

Source locator: cited publication, lines 132-134

Verbatim paper-source connection

h. Let the total utility T be the sum of all bidders' utilities. T is an obvious upper bound on the revenue that can be obtained from this set of bidders. Let F

[Next verbatim source excerpt]

does not discourage bidders from bidding their utility value. When considering truthful auctions, we assume that $b_i = u_i$. To enable analysis of auction revenue we define several properties of an input set of bids. As stated in the introduction, T is the sum of all of the bids. An equivalent definition of T is the revenue due to the optimal multi-price untruthful auction, the one that satisfies all bids at their value. The revenue for optimal fixed pricing is F . Note that F can also be

Lean-expanded paper semantic target

type:
 $\{ \text{Agent} : \text{Type } u_1 \} \rightarrow [\text{Fintype Agent}] \rightarrow (\text{Agent} \rightarrow \mathbb{R}) \rightarrow \mathbb{R}$

value:
 $\text{fun } \{ \text{Agent} : \text{Type } u_1 \} [\text{Fintype Agent}] (\text{values} : \text{Agent} \rightarrow \mathbb{R}) \Rightarrow \sum i : \text{Agent}, \text{values } i$

Recorded source-to-declaration screening Verdict: matches

Reason: The actual function sums the supplied utility-value profile over all bidders, exactly source132-134. The additional source-use displays retain this meaning; the bound truthful-reports convention applies only when identifying utilities with submitted bids.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.AdmissibleVickreySupplyRule

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.AdmissibleVickreySupplyRule

Source locator: cited publication, lines 289-291

Verbatim paper-source connection

In the unlimited supply case, taking $k = f(n)$
yields a truthful auction for any function f with
 $1 \leq f(n) < n$. This auction mechanism is not

Lean-expanded paper semantic target

type:
 $(\mathbb{N} \rightarrow \mathbb{N}) \rightarrow \text{Prop}$

value:
 $\text{fun } (f : \mathbb{N} \rightarrow \mathbb{N}) \Rightarrow \forall (n : \mathbb{N}), 2 \leq n \rightarrow 1 \leq f n \wedge f n < n$

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate permits a supply rule $f(n)$ precisely with $1 \leq f(n) < n$ for admissible populations $n \geq 2$. This is the source289-291 unlimited Vickrey family; populations below two cannot satisfy its printed supply inequalities.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.BidProfile

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.BidProfile

Source locator: cited publication, lines 195-196

Verbatim paper-source connection

We consider auctions with n bidders, each bidder i having a utility value u_i and bidding b_i . We also assume that the bids are ordered so that $b_i \leq b_{i+1}$. In

Lean-expanded paper semantic target

type:
Type u_1 → Type u_1

value:
fun (Agent : Type u_1) => Agent → ℝ

Recorded source-to-declaration screening Verdict: matches

Reason: The actual submitted-bid carrier is an arbitrary real number for each bidder, matching b_i in source195-196. Utility values remain a separate profile, and source ordering/normalization are separately expressed conditions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.BidValues

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.BidValues

Source locator: cited publication, lines 195-196

Verbatim paper-source connection

We consider auctions with n bidders, each bidder i having a utility value u_i and bidding b_i . We also assume that the bids are ordered so that $b_i \leq b_{i+1}$. In

Lean-expanded paper semantic target

type:
Type $u_1 \rightarrow$ Type u_1

value:
fun (Agent : Type u_1) => Agent $\rightarrow \mathbb{R}$

Recorded source-to-declaration screening Verdict: matches

Reason: The actual utility-value carrier is an arbitrary real number for each bidder, matching u_i in source195-196. It does not equate utilities with submitted bids.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.AuctionInput

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.AuctionInput

Source locator: cited publication, lines 195-196

Verbatim paper-source connection

We consider auctions with n bidders, each bidder i having a utility value u_i and bidding b_i . We also assume that the bids are ordered so that $b_i \leq b_{i+1}$. In

Lean-elaborated paper signature

field values:

{Agent : Type u_1} →
GHW01DigitalGoods.SourceDefinitions.AuctionInput Agent → GHW01DigitalGoods.SourceDefinitions.BidValues Agent

field bids:

{Agent : Type u_1} →
GHW01DigitalGoods.SourceDefinitions.AuctionInput Agent → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.BidValues](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual input record contains separate bidder-indexed utility and bid profiles, each Agent -> real. This preserves the two quantities u_i and b_i in source195-196 without assuming truthful reporting.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.FillsAtOrBelow

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.FillsAtOrBelow

Source locator: cited publication, lines 200-203

Verbatim paper-source connection

Given a set of bids, the outcome of an auction is the subset of bids that are satisfied and a corresponding set of sale prices such that, for each winning bid bi , the associated sale price is at most bi . A deterministic auction mechanism maps sets of bids to auction

Lean-expanded paper semantic target

type:
{Agent : Type u_1} →
GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → Prop

value:
fun {Agent : Type u_1} (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)
(bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
 $\forall i \in \text{outcome.winners}, \text{outcome.salePrice } i \leq \text{bids } i$

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate says every winning bidder pays at most that bidder's submitted bid. This is exactly the constraint in source200-203, separated from the winner/price data carrier.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.IsSinglePrice

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.IsSinglePrice

Source locator: cited publication, lines 67-70

Verbatim paper-source connection

In this paper we study a class of truthful auctions for unlimited supply. We study both singleprice auctions, where every winning bidder pays the same price, and multi-price auctions, where the prices may differ. In addition to deterministic auctions, we study randomized auctions that use randomization to

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

type:

{Agent : Type u_1} → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent → Prop

value:

fun {Agent : Type u_1} (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent) =>
 ∃ (p : ℝ), ∀ i ∈ outcome.winners, outcome.salePrice i = p

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires one real price shared by every winner. This is source67-70 single-price semantics; loser prices and the vacuous no-winner case do not affect the claim.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.profit

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.profit

Source locator: cited publication, lines 215-218

Verbatim paper-source connection

We say that an auction is truthful if bidding u_i is a dominant strategy for bidder i . More specifically, let a bidder's profit be the difference between the bidder's utility value and the price the bidder pays if they win the auction, or zero if they lose. Then an auction

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent →
  GHW01DigitalGoods.SourceDefinitions.BidValues Agent → Agent → ℝ

value:
fun {Agent : Type u_1} [DecidableEq Agent] (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)
  (values : GHW01DigitalGoods.SourceDefinitions.BidValues Agent) (i : Agent) =>
  if i ∈ outcome.winners then values i - outcome.salePrice i else 0
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidValues](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual profit is true utility minus sale price on a win and zero on a loss. It preserves the source215-218 utility/report distinction and both outcome cases.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.BidIndependentRule

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.BidIndependentRule

Source locator: cited publication, lines 351-357

Verbatim paper-source connection

Let B be the set of all bids and let B_i be the set of bids without bidder i 's bid. A bid-independent auction uses a predetermined function, f , from sets of bid values to prices. The auction works as follows. Bidder i wins the auction at price $f(B_i)$ if $b_i \geq f(B_i)$.
Otherwise, the bidder loses.

Lean-expanded paper semantic target

type:
Type
value:
Multiset $\mathbb{R} \rightarrow$ GHW01DigitalGoods.SourceDefinitions.ThresholdOffer

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.ThresholdOffer](#)

Recorded source-to-declaration screening Verdict: matches
Reason: The actual bid-independent rule takes the multiset of other bid values to a price and a threshold-boundary choice. Together with otherBids and accepts, the displayed mechanism realizes source351-357 and its permitted weak/strict comparison convention.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.CompetitiveOn

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.CompetitiveOn

Source locator: cited publication, lines 266-267

Verbatim paper-source connection

We say that an auction is competitive under certain assumptions if when the assumptions hold, the auction's revenue is $\Omega(F)$, or equivalently $R/F = \Omega(1)$.

Lean-expanded paper semantic target

```
type:
{Instance : Type u_1} → (Instance → Prop) → (Instance → ℝ) → (Instance → ℝ) → Prop

value:
fun {Instance : Type u_1} (admissible : Instance → Prop) (revenue benchmark : Instance → ℝ) =>
  ∃ (c : ℝ), 0 < c ∧ ∀ (x : Instance), admissible x → c * benchmark x ≤ revenue x
```

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires one positive constant c uniformly over all admissible instances with c times benchmark $\leq$ revenue. This is source266-267 revenue $\Omega(F)$ under the stated assumptions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.DeterministicAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.DeterministicAuction

Source locator: cited publication, lines 203-210

Verbatim paper-source connection

the associated sale price is at most b_i . A deterministic auction mechanism maps sets of bids to auction outcomes. A randomized auction mechanism maps

1 Some of our results hold under weaker assumptions.

2

[form-feed in source extraction]
sets of bids to probability distributions on auction

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

type:
Type u_1 → Type u_1
value:
fun (Agent : Type u_1) =>
GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening **Verdict:** matches
Reason: The actual deterministic mechanism maps bid profiles to winner/price outcomes, as in source203-210. Outcome validity and truthfulness are separately displayed correctness predicates rather than hidden mechanism premises.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.DeterministicTruthful

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.DeterministicTruthful

Source locator: cited publication, lines 215-222

Verbatim paper-source connection

We say that an auction is truthful if bidding u_i is a dominant strategy for bidder i . More specifically, let a bidder's profit be the difference between the bidder's utility value and the price the bidder pays if they win the auction, or zero if they lose. Then an auction is truthful if a bidder's profit (or expected profit, for randomized auctions), as a function of the bidder's bid, is maximized at the bidder's utility value, for any fixed values of the other bidders' bids. Truthful

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent → Prop

value:
fun {Agent : Type u_1} [DecidableEq Agent] (auction : GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent) =>
  ∀ (values : GHW01DigitalGoods.SourceDefinitions.BidValues Agent) (i : Agent) (report : ℝ),
    (auction (Function.update values i report)).profit values i ≤ (auction values).profit values i
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.profit](#)
- [GHW01DigitalGoods.SourceDefinitions.BidValues](#)
- [GHW01DigitalGoods.SourceDefinitions.DeterministicAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual dominant-strategy inequality compares truthful reporting with every unilateral report, holding all other coordinates fixed, using utility-based profit. Quantifying the complete real profile represents arbitrary fixed other bids as required by source215-222.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.DeterministicCorrect

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.DeterministicCorrect

Source locator: cited publication, lines 79-83

Verbatim paper-source connection

We view auctions as algorithms for deciding which input bids to fill, and at what price. As with any algorithm, one needs to address the issues of correctness, efficiency, and performance. In the context of this paper, an auction is correct if the auction is truthful and fills each winning bid at or below the bid value.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent → Prop

value:
fun {Agent : Type u_1} [DecidableEq Agent] (auction : GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent) =>
  GHW01DigitalGoods.SourceDefinitions.DeterministicTruthful auction ∧
  ∀ (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent), (auction bids).FillsAtOrBelow bids
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.FillsAtOrBelow](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.DeterministicAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.DeterministicTruthful](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate conjoins dominant-strategy truthfulness with filling every winning bid at or below its bid. These are exactly the two correctness requirements in source79-83.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.DeterministicIsSinglePrice

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.DeterministicIsSinglePrice

Source locator: cited publication, lines 67-70

Verbatim paper-source connection

In this paper we study a class of truthful auctions for unlimited supply. We study both single-price auctions, where every winning bidder pays the same price, and multi-price auctions, where the prices may differ. In addition to deterministic auctions, we study randomized auctions that use randomization to

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} → GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent → Prop

value:
fun {Agent : Type u_1} (auction : GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent) =>
  ∀ (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent), (auction bids).IsSinglePrice
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.IsSinglePrice](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.DeterministicAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires a common winner price for each bid profile, allowing that price to vary across inputs. This matches the source single-price auction class.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.boundedSupplyFk

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.boundedSupplyFk

Source locator: cited publication, lines 1599-1604

Verbatim paper-source connection

not selling all of the items. Note that the definitions of truthful and competitive auctions, which we stated for the unlimited supply case, also apply to the bounded supply case. We denote by F_k the revenue for optimal fixed pricing that sells at most k items, and it is this quantity that we wish to be competitive with.

Lean-expanded paper semantic target

type:

```
{Agent : Type u_1} →  
[Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → ℝ
```

value:

```
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : Agent → ℝ)  
  (capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>  
  GHW01DigitalGoods.boundedSupplyFixedPriceBenchmark bids capacity
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.boundedSupplyFixedPriceBenchmark](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual F_k expands to the maximum of capped fixed-price revenues over input bid candidates. On the normalized source domain this is optimal fixed pricing selling at most k , as in source1599-1604.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.IsBoundedSupplyOptimalThreshold

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.IsBoundedSupplyOptimalThreshold

Source locator: cited publication, lines 1680-1683

Verbatim paper-source connection

Let opt_k be the function that, given a set of bids, For unlimited supply markets, our analysis asreturns the optimal threshold that sells k items or sumes that there is no cost for producing the items less. This function, on the entire set of bids, gives being auctioned. With the following modification we the threshold to use for F_k . The single-price sampling can also accommodate non-zero marginal costs. If

Lean-expanded paper semantic target

type:
{Agent : Type u_1} →
[Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → ℝ → Prop

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : Agent → ℝ)
 (capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) (price : ℝ) =>
 0 ≤ price ∧
 GHW01DigitalGoods.boundedSupplySinglePriceRevenue bids capacity price =
 GHW01DigitalGoods.SourceDefinitions.boundedSupplyFk bids capacity

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedSupplyFk](#)
- [GHW01DigitalGoods.boundedSupplySinglePriceRevenue](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires a nonnegative price attaining the capped fixed-price benchmark. Its revenue uses $\min(k, \text{saleCount})$, exactly the bound opt_k cap convention; it does not incorrectly require at most k threshold-eligible bids on tied profiles.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.IsKItemBoundedSupplyAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.IsKItemBoundedSupplyAuction

Source locator: cited publication, lines 1595-1600

Verbatim paper-source connection

Up to this point, we have studied the unlimited supply case which is motivated by the digital goods market where the cost of making a copy of an item is $\hookrightarrow$ negligible. In this section we consider the case where the number of items available for sale is bounded. This case is typical of physical goods markets. We denote the number of items available by k . Here again, the seller wishes to maximize their revenue, possibly not selling all of the items. Note that the definitions of truthful and competitive auctions, which we stated for the unlimited supply case, also apply to the

Lean-expanded paper semantic target

type:

{Agent : Type u_1} →

[Fintype Agent] →

AppliedModelingLib.Auction.DigitalGoodsAuction Agent → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → Prop

value:

fun {Agent : Type u_1} [Fintype Agent] (auction : AppliedModelingLib.Auction.DigitalGoodsAuction Agent)

(capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>

auction.BinaryAllocation $\wedge \forall$ (bids : Agent $\rightarrow \mathbb{R}$), $\sum i : \text{Agent}, \text{auction.allocation bids } i \leq \uparrow \text{capacity}$

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.BinaryAllocation](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires binary allocation and total allocated units at most k on every bid profile. This is the source1595-1600 bounded supply, with unsold items allowed.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyOutcome

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyOutcome

Source locator: cited publication, lines 284-287

Verbatim paper-source connection

The k-item Vickrey auction [18] was a starting place for our work. The k-item Vickrey auction is a single-price auction that sells k items to the k highest bidders at the price equal to the k + 1 highest bid (b_{n-k-1}). For generalizations of Vickrey auctions to

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.BidProfile Agent →
  ℕ → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent → Prop

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent)
  (k : ℕ) (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent) =>
  outcome.winners.card = k ∧
  (∀ i ∈ outcome.winners, ∀ j ∉ outcome.winners, bids j ≤ bids i) ∧
  ∃ critical ∉ outcome.winners,
  (∀ j ∉ outcome.winners, bids j ≤ bids critical) ∧ ∀ i ∈ outcome.winners, outcome.salePrice i = bids critical
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual outcome has exactly k winners, no loser bidding above a winner, and all winners paying the largest losing bid. This is the source k highest bidders at the next-highest price; equal bids permit the source arbitrary consistent tie order.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyAuction

Source locator: cited publication, lines 284-287

Verbatim paper-source connection

The k-item Vickrey auction [18] was a starting place for our work. The k-item Vickrey auction is a single-price auction that sells k items to the k highest bidders at the price equal to the k + 1 highest bid (b_{n-k-1}). For generalizations of Vickrey auctions to

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[FIntype Agent] → [DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent → ℕ → Prop

value:
fun {Agent : Type u_1} [FIntype Agent] [DecidableEq Agent]
(auction : GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent) (k : ℕ) =>
  ∀ (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent),
    GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyOutcome bids k (auction bids)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.DeterministicAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual auction predicate applies the complete k-item Vickrey outcome condition to every bid profile. The displayed outcome requires k highest winners and a common highest-losing-bid price, preserving source284-287.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue

Source locator: cited publication, lines 97-103

Verbatim paper-source connection

One way to set prices for items in unlimited supply is to estimate consumer utility via market analysis and then set a fixed price. We refer to this method as fixed pricing. Pay-per-view movies are an example of fixed pricing for an unlimited supply market. With perfect knowledge of consumer utilities, optimal fixed pricing maximizes fixed-price revenue by selecting the optimal price at which to sell items. Fixed pricing generally cannot achieve this ideal due to the

Lean-expanded paper semantic target

type:
{Agent : Type u_1} → [Fintype Agent] → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → ℝ → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) (p : ℝ) =>
AppliedModelingLib.Auction.singlePriceRevenue bids p

Direct retained dependencies

- [AppliedModelingLib.Auction.singlePriceRevenue](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual revenue is the finite sum of p over bids accepting p and zero otherwise. This is the revenue of the displayed fixed-price outcome and the source fixed-pricing objective.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue

Source locator: cited publication, lines 134-138

Verbatim paper-source connection

that can be obtained from this set of bidders. Let F be the optimal fixed pricing revenue. Clearly $F \leq T$. F is an upper bound on the revenue that can be obtained by any fixed priced sale or any single-price auction. We want revenues of our truthful auctions

Lean-expanded paper semantic target

type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
Finset.univ.sup' GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue._proof_1 fun (i : Agent) =>
GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue bids (bids i)

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual F is the maximum weak fixed-price revenue over submitted bid values. On the source normalized domain this attains the source134-138 optimal fixed-price benchmark; the bound normalization convention excludes non-source fallback interpretations.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.IsOptimalThreshold

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.IsOptimalThreshold

Source locator: cited publication, lines 317-335

Verbatim paper-source connection

To get a better understanding of how the optimal single-price auction works, we define the optimal threshold function, $\text{opt}(B)$. This function on a set of bids B returns the fixed price at which items should be sold to achieve revenue F . In the optimal single-price auction, all bid values that are at least $\text{opt}(B)$ will be satisfied at price $\text{opt}(B)$. All other bids will be rejected. That is,

5

Generalized Truthful Auction Mechanisms

By making observations about auctions that encourage utility value bids, in particular the Vickrey auction, we can design general auction mechanisms. We present two general auction mechanisms that facilitate the design of truthful auctions. The first, the bid-independent auction mechanism, is based on the

→ observation that the price that a bid is satisfied at must be independent from that bid's actual value. The

second, the random sampling auction mechanism, is based on the observation that rejected bids can be used to set prices for bids that are to be satisfied.

$\text{opt}(B) = \text{argmax}_{b_i \in B} b_i \cdot (n - i + 1)$.

Lean-expanded paper semantic target

type:

$\{\text{Agent} : \text{Type } u_1\} \rightarrow$
 $[\text{Fintype Agent}] \rightarrow [\text{Nonempty Agent}] \rightarrow \text{GHW01DigitalGoods.SourceDefinitions.BidProfile Agent} \rightarrow \mathbb{R} \rightarrow \text{Prop}$

value:

fun $\{\text{Agent} : \text{Type } u_1\} [\text{Fintype Agent}] [\text{Nonempty Agent}] (\text{bids} : \text{GHW01DigitalGoods.SourceDefinitions.BidProfile Agent})$
 $(p : \mathbb{R}) =>$
 $(\exists (i : \text{Agent}), p = \text{bids } i) \wedge$
 $\text{GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue bids } p =$
 $\text{GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue bids}$

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue](#)
- [GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires that the threshold is an input bid and attains the optimal fixed-price revenue. This is source317-335 argmax over submitted bid values, with weak acceptance at the chosen price.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.lowestBid

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.lowestBid

Source locator: cited publication, lines 130-132

Verbatim paper-source connection

bidders. Without loss of generality, we assume that the lowest bid is one and denote the highest bid by h . Let the total utility T be the sum of all bidders'

Lean-expanded paper semantic target

type:

$\{ \text{Agent} : \text{Type } u_1 \} \rightarrow [\text{Fintype Agent}] \rightarrow [\text{Nonempty Agent}] \rightarrow \text{GHW01DigitalGoods.SourceDefinitions.BidProfile Agent} \rightarrow \mathbb{R}$

value:

$\text{fun } \{ \text{Agent} : \text{Type } u_1 \} [\text{Fintype Agent}] [\text{Nonempty Agent}] (\text{bids} : \text{GHW01DigitalGoods.SourceDefinitions.BidProfile Agent}) \Rightarrow$
 $\text{Finset.univ.inf } \text{GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue_proof_1 bids}$

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual lowest bid is the finite minimum over a nonempty bid profile, matching the quantity normalized to one in source130-132.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.NormalizedBidScale

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.NormalizedBidScale

Source locator: cited publication, lines 130-132

Verbatim paper-source connection

bidders. Without loss of generality, we assume that the lowest bid is one and denote the highest bid by h . Let the total utility T be the sum of all bidders'

Lean-expanded paper semantic target

type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → Prop

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
GHW01DigitalGoods.SourceDefinitions.lowestBid bids = 1

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.lowestBid](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual normalized-scale predicate sets the minimum submitted bid to one, exactly source130-132. Nonempty finite input is explicit for the minimum.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.PositiveAlpha

Source locator: cited publication, lines 257-265

Verbatim paper-source connection

can make the assumption that the optimal revenue F is significantly larger than h , the highest bid. That is, for some constant α , we assume that $\alpha h \leq F$. With this assumption, optimal fixed pricing sells at least α items. In some cases α is a fixed constant. In other cases, success probability approaches 1 as $\alpha \rightarrow \infty$. For some proofs, this assumption is stronger than what we need and we make the weaker assumption that $\alpha h \leq T$.

Lean-expanded paper semantic target

type:
 $\mathbb{R} \rightarrow \text{Prop}$

 value:
 fun (alpha : $\mathbb{R}$) => 0 < alpha

Recorded source-to-declaration screening Verdict: matches
 Reason: The actual alpha condition is strict positivity, the constant-scale domain in source257-265. The two substantive scale inequalities are separately displayed functions.
 Reviewer check: ☐ Matches source input
 If left unchecked, explain the discrepancy or uncertainty below.
 Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.directionalRealization

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.directionalRealization

Source locator: cited publication, lines 410-416

Verbatim paper-source connection

Another general truthful auction mechanism is based on randomized sampling. We select a subset B' of B at random, independent of the bid values. Let m be the size of B' . We then compute a function on these sampled bids, $f(B')$, and use this as a threshold value for the $n-m$ bids in the non-sample, $B \setminus B'$. Note that this auction mechanism is inherently single-price.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent →
  Finset Agent → AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [DecidableEq Agent] (plan : GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent)
  (sample : Finset Agent) =>
  GHW01DigitalGoods.SourceDefinitions.randomSamplingAuction sample plan.threshold
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan](#)
- [GHW01DigitalGoods.SourceDefinitions.randomSamplingAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual realized directional mechanism uses only the selected sample's bid multiset to price the nonsample and assigns sample bidders zero allocation/payment. The plan fixes both sampling and threshold function independently of the bid profile, matching source410-416.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.directionalOutcomeLaw

Source locator: cited publication, lines 410-416

Verbatim paper-source connection

Another general truthful auction mechanism is based on randomized sampling. We select a subset B' of B at random, independent of the bid values. Let m be the size of B' . We then compute a function on these sampled bids, $f(B')$, and use this as a threshold value for the $n-m$ bids in the non-sample, $B \setminus B'$. Note that this auction mechanism is inherently single-price.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent →
  (Agent → ℝ) → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent]
  (plan : GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent) (bids : Agent → ℝ) =>
  PMF.map
    (fun (sample : Finset Agent) =>
      GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome (plan.directionalRealization sample) bids)
  plan.sampleLaw
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.directionalRealization](#)
- [GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual directional outcome law pushes the bid-independent sample PMF through the directional auction realization. Sample bidders are rejected and only complement bidders receive the sample-derived threshold, as in source410-416.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.dualPriceRealization

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.dualPriceRealization

Source locator: cited publication, lines 417-422

Verbatim paper-source connection

If a multi-price auction is acceptable, then this auction can be modified to be dual-price with $m \approx n/2$, by using $f(B')$ to compute the threshold to use for bids in $B \setminus B'$ and $f(B \setminus B')$ as the threshold to use for bids in B' . This is a good way to avoid revenue loss due to the rejected bids in the sample; however, it is at the expense of making the auction dual-price.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent →
  Finset Agent → AppliedModelingLib.Auction.DigitalGoodsAuction Agent
value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent]
  (plan : GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent) (sample : Finset Agent) =>
  GHW01DigitalGoods.SourceDefinitions.dualPriceRandomSamplingAuction sample plan.threshold
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan](#)
- [GHW01DigitalGoods.SourceDefinitions.dualPriceRandomSamplingAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual dual-price realization computes f on the complement for sampled bidders and f on the sample for nonsampled bidders, charging the corresponding threshold on acceptance. This preserves both directions in source417-422.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.dualPriceOutcomeLaw

Source locator: cited publication, lines 417-422

Verbatim paper-source connection

If a multi-price auction is acceptable, then this auction can be modified to be dual-price with $m \approx n/2$, by using $f(B')$ to compute the threshold to use for bids in $B \setminus B'$ and $f(B \setminus B')$ as the threshold to use for bids in B' . This is a good way to avoid revenue loss due to the rejected bids in the sample; however, it is at the expense of making the auction dual-price.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent →
  (Agent → ℝ) → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent]
  (plan : GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan Agent) (bids : Agent → ℝ) =>
  PMF.map
    (fun (sample : Finset Agent) =>
      GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome (plan.dualPriceRealization sample) bids)
  plan.sampleLaw
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.dualPriceRealization](#)
- [GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual dual-price law pushes the same bid-independent sample PMF through the two-sided realization. Each side receives the opposite side's threshold, exactly source417-422.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.RandomizedAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomizedAuction

Source locator: cited publication, lines 203-210

Verbatim paper-source connection

the associated sale price is at most b_i . A deterministic auction mechanism maps sets of bids to auction outcomes. A randomized auction mechanism maps

1 Some of our results hold under weaker assumptions.

2

[form-feed in source extraction]

sets of bids to probability distributions on auction

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

type:

Type $u_1 \rightarrow \text{Type } u_1$

value:

fun (Agent : Type u_1) =>

GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening **Verdict:** matches

Reason: The actual randomized mechanism maps bids to PMFs of winner/price outcomes. It matches source203-210 under the explicitly bound discrete/countably-supported outcome-law convention, without claiming to represent atomless outcome distributions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.RandomizedTruthful

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomizedTruthful

Source locator: cited publication, lines 215-222

Verbatim paper-source connection

We say that an auction is truthful if bidding u_i is a dominant strategy for bidder i . More specifically, let a bidder's profit be the difference between the bidder's utility value and the price the bidder pays if they win the auction, or zero if they lose. Then an auction is truthful if a bidder's profit (or expected profit, for randomized auctions), as a function of the bidder's bid, is maximized at the bidder's utility value, for any fixed values of the other bidders' bids. Truthful

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent → Prop

value:
fun {Agent : Type u_1} [DecidableEq Agent] (auction : GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent) =>
  GHW01DigitalGoods.SourceDefinitions.HasFiniteExpectedProfits auction ∧
  ∀ (values : GHW01DigitalGoods.SourceDefinitions.BidValues Agent) (i : Agent) (report : ℝ),
    GHW01DigitalGoods.SourceDefinitions.randomizedExpectedProfit auction values i (Function.update values i report) ≤
    GHW01DigitalGoods.SourceDefinitions.randomizedExpectedProfit auction values i values
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidValues](#)
- [GHW01DigitalGoods.SourceDefinitions.HasFiniteExpectedProfits](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomizedAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.randomizedExpectedProfit](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate first requires summable weighted profit series and then compares expected truthful profit with every unilateral report. It preserves source215-222 under the explicitly bound PMF-domain and finite-real-expectation conventions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.RandomizedCorrect

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomizedCorrect

Source locator: cited publication, lines 79-83

Verbatim paper-source connection

We view auctions as algorithms for deciding which input bids to fill, and at what price. As with any algorithm, one needs to address the issues of correctness, efficiency, and performance. In the context of this paper, an auction is correct if the auction is truthful and fills each winning bid at or below the bid value.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} → [DecidableEq Agent] → GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent → Prop

value:
fun {Agent : Type u_1} [DecidableEq Agent] (auction : GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent) =>
  GHW01DigitalGoods.SourceDefinitions.RandomizedTruthful auction ∧
  ∀ (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent)
    (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent),
    (auction bids) outcome ≠ 0 → outcome.FillsAtOrBelow bids
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.FillsAtOrBelow](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomizedAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomizedTruthful](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual randomized correctness predicate combines finite-expectation dominant-strategy truthfulness with the winner-price constraint on every positive-mass outcome. This matches source79-83 under the bound PMF and expected-profit-finiteness conventions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.RandomizedIsSinglePrice

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RandomizedIsSinglePrice

Source locator: cited publication, lines 67-70

Verbatim paper-source connection

In this paper we study a class of truthful auctions for unlimited supply. We study both singleprice auctions, where every winning bidder pays the same price, and multi-price auctions, where the prices may differ. In addition to deterministic auctions, we study randomized auctions that use randomization to

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} → GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent → Prop

value:
fun {Agent : Type u_1} (auction : GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent) =>
  ∀ (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent)
    (outcome : GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent),
    (auction bids) outcome ≠ 0 → outcome.IsSinglePrice
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.IsSinglePrice](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomizedAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires every positive-mass outcome to have one common winner price. This is randomized single-price semantics under the bound discrete outcome-law convention, not a requirement that all random outcomes share the same price.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RevenueBigO

Source locator: cited publication, lines 239-245

Verbatim paper-source connection

Analogous to on-line algorithm theory, we express auction performance relative to that for optimal untruthful auctions, as ratios R/T or R/F . However, we solve a maximization problem while on-line algorithms solve minimization problems. Thus, positive results, which are lower bounds on R/T or R/F , are expressed using “ Ω ”. Impossibility results, which are upper bounds on R/T or R/F for any auction in a certain class, are expressed using “ O ”.

Lean-expanded paper semantic target

type:
{Instance : Type u_1} → (Instance → ℝ) → (Instance → ℝ) → Prop

value:
fun {Instance : Type u_1} (revenue benchmark : Instance → ℝ) =>
 ∃ (C : ℝ), 0 ≤ C ∧ ∀ (x : Instance), revenue x ≤ C * benchmark x

Recorded source-to-declaration screening Verdict: matches

Reason: The actual definition gives a uniform nonnegative constant upper bound revenue ≤ C times benchmark on the instance family. This is the source239-245 revenue-upper-bound performance convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.RevenueBigOmega

Source locator: cited publication, lines 239-245

Verbatim paper-source connection

Analogous to on-line algorithm theory, we express auction performance relative to that for optimal untruthful auctions, as ratios R/T or R/F . However, we solve a maximization problem while on-line algorithms solve minimization problems. Thus, positive results, which are lower bounds on R/T or R/F , are expressed using “ Ω ”. Impossibility results, which are upper bounds on R/T or R/F for any auction in a certain class, are expressed using “ O ”.

Lean-expanded paper semantic target

```
type:
{Instance : Type u_1} → (Instance → ℝ) → (Instance → ℝ) → Prop

value:
fun {Instance : Type u_1} (revenue benchmark : Instance → ℝ) =>
  ∃ (c : ℝ), 0 < c ∧ ∀ (x : Instance), c * benchmark x ≤ revenue x
```

Recorded source-to-declaration screening Verdict: matches

Reason: The actual definition gives a uniform positive constant lower bound c times benchmark $\leq$ revenue on the instance family. This is the source239-245 revenue-lower-bound convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.SortedBids

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.SortedBids

Source locator: cited publication, lines 196-199

Verbatim paper-source connection

having a utility value u_i and bidding b_i . We also assume that the bids are ordered so that $b_i \leq b_{i+1}$. In auctions where ties need to be broken, we can assume an arbitrary total order on the bid values that respects the partial order. That is, we can assume that the order given by the indices is strict. We assume that there is no collusion among the bidders.

Lean-expanded paper semantic target

```
type:
{n : ℕ} → GHW01DigitalGoods.SourceDefinitions.BidProfile (Fin n) → Prop

value:
fun {n : ℕ} (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile (Fin n)) =>
  ∀ (i j : Fin n), ↑i ≤ ↑j → bids i ≤ bids j
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate orders the finite indexed bids nondecreasingly. This is the numerical-order component of source196-199; deterministic allocation tie selection is separately represented where needed.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary

Source locator: cited publication, lines 372-374

Verbatim paper-source connection

2 For maximum generality, we allow f to also return whether to use $\geq$ or $>$ in this comparison. For the purpose of maximizing revenue we can always just use $\geq$; however, to prove Lemma 9.2 we need the full generality.

Lean-elaborated paper signature

type:
Type

constructor GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary.weak:
GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary

constructor GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary.strict:
GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary

Recorded source-to-declaration screening Verdict: matches

Reason: The actual two-constructor boundary type records the source372-374 choice between accepting equality and requiring strict inequality. The displayed accepts function supplies those exact comparisons.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.TruthfulReports

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.TruthfulReports

Source locator: cited publication, lines 132-134

Verbatim paper-source connection

h. Let the total utility T be the sum of all bidders' utilities. T is an obvious upper bound on the revenue that can be obtained from this set of bidders. Let F

[Next verbatim source excerpt]

does not discourage bidders from bidding their utility value. When considering truthful auctions, we assume that $b_i = u_i$. To enable analysis of auction revenue we define several properties of an input set of bids. As stated in the introduction, T is the sum of all of the bids. An equivalent definition of T is the revenue due to the optimal multi-price untruthful auction, the one that satisfies all bids at their value. The revenue for optimal fixed pricing is F . Note that F can also be

Lean-expanded paper semantic target

type:
{Agent : Type u_1} → GHW01DigitalGoods.SourceDefinitions.AuctionInput Agent → Prop

value:
fun {Agent : Type u_1} (input : GHW01DigitalGoods.SourceDefinitions.AuctionInput Agent) => input.bids = input.values

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionInput](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate equates submitted bids with utility values. This is source226-232 and the bound convention governing only the bridge between utility total and bid total.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyDeterministicAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyDeterministicAuction

Source locator: cited publication, lines 89-96

Verbatim paper-source connection

Consider the problem of selling a number of identical items to consumers who each want a single item and the items are available in unlimited supply. By unlimited supply we mean that either the seller has at least as many items as there are consumers, or that the seller can reproduce items on demand at negligible marginal cost. Of particular interest are digital items such as downloadable audio files and ⇐ payper-view movies. With unlimited supply, consumer utilities, the maximum amounts that consumers are willing to pay for the item, are the sole factor determining sale prices. The seller's goal is to maximize their total revenue.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

type:
Type u_1 → Type u_1

value:
fun (Agent : Type u_1) => GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.DeterministicAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual unlimited-supply deterministic carrier adds no shared capacity bound to the per-bidder outcome mechanism. This represents source89-96 ability to serve any subset when at least one item per bidder is available.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyRandomizedAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyRandomizedAuction

Source locator: cited publication, lines 89-96

Verbatim paper-source connection

Consider the problem of selling a number of identical items to consumers who each want a single item and the items are available in unlimited supply. By unlimited supply we mean that either the seller has at least as many items as there are consumers, or that the seller can reproduce items on demand at negligible marginal cost. Of particular interest are digital items such as downloadable audio files and ⇐ payer-view movies. With unlimited supply, consumer utilities, the maximum amounts that consumers are willing to pay for the item, are the sole factor determining sale prices. The seller's goal is to maximize their total revenue.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

type:
Type u_1 → Type u_1

value:
fun (Agent : Type u_1) => GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.RandomizedAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual unlimited-supply randomized carrier adds no shared capacity restriction to its outcome PMF. This matches source89-96 under the explicitly bound discrete-randomization convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.accepts

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.accepts

Source locator: cited publication, lines 372-374

Verbatim paper-source connection

2 For maximum generality, we allow f to also return whether to use $\geq$ or $>$ in this comparison. For the purpose of maximizing revenue we can always just use $\geq$; however, to prove Lemma 9.2 we need the full generality.

Lean-expanded paper semantic target

type:

GHW01DigitalGoods.SourceDefinitions.ThresholdOffer $\rightarrow \mathbb{R} \rightarrow \text{Prop}$

value:

fun (offer : GHW01DigitalGoods.SourceDefinitions.ThresholdOffer) (bid : $\mathbb{R}$) =>

match offer.boundary with
| GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary.weak => offer.price $\leq$ bid
| GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary.strict => offer.price < bid

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdOffer](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual acceptance function is price $\leq$ bid for the weak boundary and price < bid for the strict boundary. These are exactly the alternatives allowed in source372-374.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.bidIndependentOutcome

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.bidIndependentOutcome

Source locator: cited publication, lines 351-357

Verbatim paper-source connection

Let B be the set of all bids and let B_i be the set of bids without bidder i 's bid. A bid-independent auction uses a predetermined function, f , from sets of bid values to prices. The auction works as follows. Bidder i wins the auction at price $f(B_i)$ if $b_i \geq f(B_i)$.
Otherwise, the bidder loses.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[Fintype Agent] →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.BidIndependentRule →
  GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent]
  (rule : GHW01DigitalGoods.SourceDefinitions.BidIndependentRule)
  (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
{
  winners :=
  {i : Agent |
    GHW01DigitalGoods.SourceDefinitions.accepts (rule (GHW01DigitalGoods.SourceDefinitions.otherBids bids i))
    (bids i)},
  salePrice := fun (i : Agent) => (rule (GHW01DigitalGoods.SourceDefinitions.otherBids bids i)).price }
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidIndependentRule](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdOffer](#)
- [GHW01DigitalGoods.SourceDefinitions.accepts](#)
- [GHW01DigitalGoods.SourceDefinitions.otherBids](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual winners are exactly bidders satisfying their own-erased offer; winner sale prices are the corresponding $f(B_i)$ prices. The displayed otherBids preserves the other bids with multiplicity and excludes self, matching source351-357.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.bidIndependentAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.bidIndependentAuction

Source locator: cited publication, lines 351-357

Verbatim paper-source connection

Let B be the set of all bids and let B_i be the set of bids without bidder i 's bid. A bid-independent auction uses a predetermined function, f , from sets of bid values to prices. The auction works as follows. Bidder i wins the auction at price $f(B_i)$ if $b_i \geq f(B_i)$.
Otherwise, the bidder loses.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[FIntype Agent] →
[DecidableEq Agent] →
  GHW01DigitalGoods.SourceDefinitions.BidIndependentRule →
  GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent

value:
fun {Agent : Type u_1} [FIntype Agent] [DecidableEq Agent]
  (rule : GHW01DigitalGoods.SourceDefinitions.BidIndependentRule)
  (a : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
  GHW01DigitalGoods.SourceDefinitions.bidIndependentOutcome rule a
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidIndependentRule](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.bidIndependentOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual auction applies the bid-independent outcome function to the submitted profile. The displayed otherBids removes the bidder's own coordinate before the common rule chooses its threshold, matching source351-357.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.bidderCount

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.bidderCount

Source locator: cited publication, lines 128-130

Verbatim paper-source connection

To state our results more formally, we introduce the following notation. Let n denote the number of bidders. Without loss of generality, we assume that

Lean-expanded paper semantic target

type:
(Agent : Type u_1) → [Fintype Agent] → $\mathbb{N}$

value:
fun (Agent : Type u_1) [Fintype Agent] => Fintype.card Agent

Recorded source-to-declaration screening Verdict: matches

Reason: The actual bidder count is the cardinality of the finite bidder type, exactly n in source128-130.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.boundedDualPriceSampling

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.boundedDualPriceSampling

Source locator: cited publication, lines 1692-1696

Verbatim paper-source connection

In the dual-price auction with the sample size of costs. In this case the opt function would, as above $m = n/2$, use $\text{opt}k/2$ so that about $k/2$ bids are se- for bounded supply, need to be parameterized by the lected from each of the sample and the non-sample. ratio of the sample size and the non-sample size so as Once can show that the resulting auction is also to correctly use marginal cost information. truthful and competitive.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[Nonempty Agent] →
[inst_2 : LinearOrder Agent] →
(sampleSize : ℕ) →
0 < sampleSize →
Fintype.card Agent = sampleSize + sampleSize →
↑ (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize) →
GHW01DigitalGoods.SourceDefinitions.SupplyCapacity →
AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [LinearOrder Agent] (sampleSize : ℕ)
(hsample_pos : 0 < sampleSize) (hhalf : Fintype.card Agent = sampleSize + sampleSize)
(sample : ↑ (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize))
(halfCapacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>
GHW01DigitalGoods.SourceDefinitions.canonicalBoundedDualPriceSamplingAuction (↑ sample)
(GHW01DigitalGoods.SourceDefinitions.boundedDualPriceSampling._proof_1 sampleSize hsample_pos sample)
(GHW01DigitalGoods.SourceDefinitions.boundedDualPriceSampling._proof_2 sampleSize hsample_pos hhalf sample)
halfCapacity
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.canonicalBoundedDualPriceSamplingAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual realized bounded dual auction uses nonempty exact-half sample and complement, allocates at most q on each side at the opposite side's optimal q -capacity price, and uses fixed bidder priority for excess acceptors. This matches source1692-1696 under its bound total-capacity- $2q$ convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.boundedDualExactHalfOutcomeLaw

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.boundedDualExactHalfOutcomeLaw

Source locator: cited publication, lines 1692-1696

Verbatim paper-source connection

In the dual-price auction with the sample size of costs. In this case the opt function would, as above $m = n/2$, use $\text{opt}k/2$ so that about $k/2$ bids are selected from each of the sample and the non-sample. ratio of the sample size and the non-sample size so as Once can show that the resulting auction is also to correctly use marginal cost information. truthful and competitive.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
  [inst : Fintype Agent] →
    [Nonempty Agent] →
      [LinearOrder Agent] →
        (sampleSize : ℕ) →
          0 < sampleSize →
            Fintype.card Agent = sampleSize + sampleSize →
              GHW01DigitalGoods.SourceDefinitions.SupplyCapacity →
                (Agent → ℝ) → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [LinearOrder Agent] (sampleSize : ℕ)
  (hsample_pos : 0 < sampleSize) (hhalf : Fintype.card Agent = sampleSize + sampleSize)
  (halfCapacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) (bids : Agent → ℝ) =>
  PMF.map
    (fun (sample : 1 (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize)) =>
      GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome
        (GHW01DigitalGoods.SourceDefinitions.boundedDualPriceSampling sampleSize hsample_pos hhalf sample halfCapacity)
        bids)
    (GHW01DigitalGoods.SourceDefinitions.exactHalfSampleLaw Agent sampleSize hsample_pos hhalf)
```

Direct retained dependencies

- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedDualPriceSampling](#)
- [GHW01DigitalGoods.SourceDefinitions.exactHalfSampleLaw](#)
- [GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual PMF samples a uniformly random exact-half partition and maps it to the bounded two-sided auction. Each side has halfCapacity and uses the opposite side's opt_halfCapacity threshold, under the bound even-capacity and exogenous-priority conventions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceFixedSizeOutcomeLaw

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceFixedSizeOutcomeLaw

Source locator: cited publication, lines 1679-1690

Verbatim paper-source connection

We now show how to extend our optimal threshold simulations, we have argued that our auctions comsampling auctions to the k-item bounded supply

↪ case. pare favorably to fixed pricing with market analysis.

Let optk be the function that, given a set of bids,

For unlimited supply markets, our analysis asreturns the optimal threshold that sells k items or sumes that there is no cost for producing the items

less. This function, on the entire set of bids, gives being auctioned. With the following modification we

the threshold to use for F_k . The single-price sampling can also accommodate non-zero marginal costs. If optimal threshold mechanisms can now be modified the marginal cost is v per item, then first subtract v

to use threshold function $\text{optmk}/(n-m)$ on sample of from each bid and reject all negative value bids. After

size m . If this results in too many bids being satisfied, running the auction, add v back to the selling price

arbitrarily (e.g. at random) reject bids until there are of all winning bids. If the marginal cost of producing

only k left. One can show that with high probability, k items is a more complicated function of k , we can

the number of bids rejected this way will be small modify the opt function used in the sampling optimal

and that this auction is competitive.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[Nonempty Agent] →
[LinearOrder Agent] →
(sampleSize : ℕ) →
0 < sampleSize →
sampleSize < Fintype.card Agent →
GHW01DigitalGoods.SourceDefinitions.SupplyCapacity →
(Agent → ℝ) → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [LinearOrder Agent] (sampleSize : ℕ)
(hsample_pos : 0 < sampleSize) (hsample_lt : sampleSize < Fintype.card Agent)
(capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) (bids : Agent → ℝ) =>
GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceCertifiedOutcomeLaw
(PMF.map (GHW01DigitalGoods.SourceDefinitions.certifiedFixedSizePartition sampleSize hsample_pos hsample_lt)
(GHW01DigitalGoods.uniformFixedSizeSampleLaw sampleSize
(GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceFixedSizeOutcomeLaw._proof_1 sampleSize hsample_lt)))
capacity bids
```

Direct retained dependencies

- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.CertifiedFixedPartition](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceCertifiedOutcomeLaw](#)
- [GHW01DigitalGoods.SourceDefinitions.certifiedFixedSizePartition](#)
- [GHW01DigitalGoods.uniformFixedSizeSampleLaw](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual law samples uniformly among size- m subsets with $0 < m < n$, derives the certified partition, and pushes it through the capped directional auction. Its sample capacity and excess-winner handling are the bound $\text{floor}(mk/(n-m))$ and exogenous-priority choices for source1679-1690.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceSampling

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceSampling

Source locator: cited publication, lines 1679-1690

Verbatim paper-source connection

We now show how to extend our optimal threshold simulations, we have argued that our auctions comsampling auctions to the k-item bounded supply

→ case. pare favorably to fixed pricing with market analysis.

Let optk be the function that, given a set of bids,

For unlimited supply markets, our analysis asreturns the optimal threshold that sells k items or sumes that there is no cost for producing the items

less. This function, on the entire set of bids, gives being auctioned. With the following modification we

the threshold to use for F_k . The single-price sampling can also accommodate non-zero marginal costs. If optimal threshold mechanisms can now be modified the marginal cost is v per item, then first subtract v to use threshold function $\text{optmk}/(n-m)$ on sample of from each bid and reject all negative value bids. After

size m . If this results in too many bids being satisfied, running the auction, add v back to the selling price

arbitrarily (e.g. at random) reject bids until there are of all winning bids. If the marginal cost of producing

only k left. One can show that with high probability, k items is a more complicated function of k , we can

the number of bids rejected this way will be small modify the opt function used in the sampling optimal

and that this auction is competitive.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
  [inst : Fintype Agent] →
    [Nonempty Agent] →
      [LinearOrder Agent] →
        (side : Agent → Bool) →
          (sampleSide : Bool) →
            GHW01DigitalGoods.SourceDefinitions.SupplyCapacity →
              0 < GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side sampleSide →
                0 < GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side !sampleSide →
                  AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [LinearOrder Agent] (side : Agent → Bool) (sampleSide : Bool)
  (capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity)
  (hsample_pos : 0 < GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side sampleSide)
  (hmarket_pos : 0 < GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide side !sampleSide) =>
  GHW01DigitalGoods.boundedCappedThresholdAuction side (!sampleSide)
  (GHW01DigitalGoods.SourceDefinitions.canonicalBoundedSamplingPriceRule side sampleSide
   (GHW01DigitalGoods.SourceDefinitions.boundedSampleCapacityFloor side sampleSide capacity) hsample_pos)
  capacity
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.bidderCountOnSide](#)
- [GHW01DigitalGoods.SourceDefinitions.boundedSampleCapacityFloor](#)
- [GHW01DigitalGoods.SourceDefinitions.canonicalBoundedSamplingPriceRule](#)
- [GHW01DigitalGoods.boundedCappedThresholdAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual directional bounded mechanism prices only the opposite side from the nonempty sampled side's $\text{opt_floor}(\text{mk}/(n-m))$ and caps market winners at k . Both actual side cardinalities and fixed exogenous priority are displayed, matching the bound source1679-1690 conventions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.boundedSupplyOptK

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.boundedSupplyOptK

Source locator: cited publication, lines 1680-1683

Verbatim paper-source connection

Let opt_k be the function that, given a set of bids, For unlimited supply markets, our analysis asreturns the optimal threshold that sells k items or sumes that there is no cost for producing the items less. This function, on the entire set of bids, gives being auctioned. With the following modification we the threshold to use for F_k . The single-price sampling can also accommodate non-zero marginal costs. If

Lean-expanded paper semantic target

type:
{Agent : Type u_1} →
[Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : Agent → ℝ)
(capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>
GHW01DigitalGoods.SourceDefinitions.canonicalBoundedOptimalThreshold bids capacity

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.canonicalBoundedOptimalThreshold](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual opt_k selects a maximizing nonnegative candidate price for $\min(k, \text{saleCount})$ times price. This is the bound capped-sales interpretation of source1680-1683, including tied profiles.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.boundedSupplyTk

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.boundedSupplyTk

Source locator: cited publication, lines 1629-1631

Verbatim paper-source connection

to the optimal untruthful multi-price auction. Define T_k to be the revenue of the optimal multi-price auction restricted to only satisfying k bidders (i.e. the sum of the highest k bids) and F_k as above. Then

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[DecidableEq Agent] →
(Agent → ℝ) → (capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) → capacity ≤ Fintype.card Agent → ℝ

value:
fun {Agent : Type u_1} [Fintype Agent] [DecidableEq Agent] (bids : Agent → ℝ)
(capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) (hcapacity : capacity ≤ Fintype.card Agent) =>
{s ∈ Finset.univ.powerset | s.card = capacity}.sup'
(GHW01DigitalGoods.SourceDefinitions.boundedSupplyTk._proof_1 capacity hcapacity) fun (s : Finset Agent) =>
∑ i ∈ s, bids i
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual T_k is the maximum total bid over subsets of exactly k bidders, with $k \leq n$ explicit. This directly implements source1629-1631 sum of the highest k bids and does not substitute an unrestricted at-most- k maximum.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.deterministicOptimalThresholdAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.deterministicOptimalThresholdAuction

Source locator: cited publication, lines 991-1013

Verbatim paper-source connection

The deterministic optimal threshold auction is the
are satisfied at price h . Other values of f lead to
bid-independent auction with $f = \text{opt}$, the optimal
smaller revenues.
threshold function defined in Section 4. The only difWe wish to find an input family such that the
ference between the deterministic optimal threshold
bid-independent
auction with function f has revenue
auction and the optimal fixed pricing mechanism is
 $O(F$
 $/h)$.
To
obtain
this family, we chose n_h and n_{ell} in
that the former uses threshold $\text{opt}(B_i)$ for bidder i
such
a
way
that
 f
(n
 $h - 1, n_{ell}) = 1$, $f(n_h, n_{ell} - 1) = h$,
and the latter uses $\text{opt}(B)$. Recall that B_i and B

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[Nonempty Agent] →
[DecidableEq Agent] → 2 ≤ Fintype.card Agent → AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [DecidableEq Agent] (hcard : 2 ≤ Fintype.card Agent) =>
  AppliedModelingLib.Auction.thresholdPriceAuction fun (bids : Agent → ℝ) (i : Agent) =>
    GHW01DigitalGoods.SourceDefinitions.optimalThresholdOnFinset (Finset.univ.erase i)
    (GHW01DigitalGoods.SourceDefinitions.erase_univ_nonempty_of_card_ge_two hcard i) bids
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.thresholdPriceAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.optimalThresholdOnFinset](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual deterministic mechanism sets each bidder's weak threshold to opt on the actual erased finite bid set. The visible $n \geq 2$ premise ensures that set is nonempty, matching source991-1013 under the bound nonempty-other-bid convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.deterministicRevenue

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.deterministicRevenue

Source locator: cited publication, lines 211-214

Verbatim paper-source connection

outcomes. We use R to denote the auction revenue for a particular auction mechanism and set of bids. R is the sum of all sale prices. For randomized auctions, R is a random variable.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
  GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent →
    GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → ℝ

value:
fun {Agent : Type u_1} (auction : GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent)
  (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
    (auction bids).revenue
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.DeterministicAuction](#)

Recorded source-to-declaration screening Verdict: matches
Reason: The actual deterministic revenue evaluates the auction outcome and sums only winner prices. This is the deterministic specialization of source211-214.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdAuction

Source locator: cited publication, lines 455-465

Verbatim paper-source connection

The random sampling optimal threshold auction takes $f = \text{opt}$, the optimal threshold function, in the random sampling auction. Intuitively we use the sample to get an idea for a good threshold value, then we apply that threshold to the remaining bids. The auction samples $m = n/2$ bids at random, computes opt of this sample, and uses this value as a threshold for the non-sample, accepting all bids above this threshold at the threshold value. In this section we show that the expected revenue of this auction is within a constant factor of F , assuming F/h is not too small.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
  [inst : Fintype Agent] →
  [Nonempty Agent] →
  [inst_2 : DecidableEq Agent] →
  (sampleSize : ℕ) →
  0 < sampleSize →
  Fintype.card Agent = sampleSize + sampleSize →
  GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary →
  ↑(GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize) →
  AppliedModelingLib.Auction.DigitalGoodsAuction Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [DecidableEq Agent] (sampleSize : ℕ)
  (hsample_pos : 0 < sampleSize) (hhalf : Fintype.card Agent = sampleSize + sampleSize)
  (boundary : GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary)
  (sample : ↑(GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize)) =>
  GHW01DigitalGoods.SourceDefinitions.directionalOptimalThresholdAuction (↑sample)
  (GHW01DigitalGoods.SourceDefinitions.exactHalfSample_nonempty hsample_pos sample) boundary
```

Direct retained dependencies

- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary](#)
- [GHW01DigitalGoods.SourceDefinitions.directionalOptimalThresholdAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual realization computes opt on the actual nonempty exact-half sample and offers it only to the complement. Its boundary parameter includes the source455-465 strict-above variant; the separately bound strict tie convention is respected in that specialization.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdOutcomeLaw

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdOutcomeLaw

Source locator: cited publication, lines 455-465

Verbatim paper-source connection

The random sampling optimal threshold auction takes $f = \text{opt}$, the optimal threshold function, in the random sampling auction. Intuitively we use the sample to get an idea for a good threshold value, then we apply that threshold to the remaining bids. The auction samples $m = n/2$ bids at random, computes opt of this sample, and uses this value as a threshold for the non-sample, accepting all bids above this threshold at the threshold value. In this section we show that the expected revenue of this auction is within a constant factor of F , assuming F/h is not too small.

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
[inst : Fintype Agent] →
[Nonempty Agent] →
[DecidableEq Agent] →
(sampleSize : ℕ) →
0 < sampleSize →
Fintype.card Agent = sampleSize + sampleSize →
GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary →
(Agent → ℝ) → PMF (GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent)

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] [DecidableEq Agent] (sampleSize : ℕ)
(hsample_pos : 0 < sampleSize) (hhalf : Fintype.card Agent = sampleSize + sampleSize)
(boundary : GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary) (bids : Agent → ℝ) =>
PMF.map
(fun (sample : 1 (GHW01DigitalGoods.FixedSizeSampleSpace Agent sampleSize)) =>
GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome
(GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdAuction sampleSize hsample_pos hhalf boundary
sample)
bids)
(GHW01DigitalGoods.SourceDefinitions.exactHalfSampleLaw Agent sampleSize hsample_pos hhalf)
```

Direct retained dependencies

- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary](#)
- [GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdAuction](#)
- [GHW01DigitalGoods.SourceDefinitions.exactHalfSampleLaw](#)
- [GHW01DigitalGoods.SourceDefinitions.realizedAuctionOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual outcome law is the uniform exact-half sample law pushed through the optimal-threshold realization. It preserves the source455-465 fixed sample size, nonempty sample, sample exclusion, and threshold-boundary parameter containing the printed strict variant.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome

Source locator: cited publication, lines 97-103

Verbatim paper-source connection

One way to set prices for items in unlimited supply is to estimate consumer utility via market analysis and then set a fixed price. We refer to this method as fixed pricing. Pay-per-view movies are an example of fixed pricing for an unlimited supply market. With perfect knowledge of consumer utilities, optimal fixed pricing maximizes fixed-price revenue by selecting the optimal price at which to sell items. Fixed pricing generally cannot achieve this ideal due to the

Lean-expanded paper semantic target

type:
{Agent : Type u_1} →
[Fintype Agent] →
GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → ℝ → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent
value:
fun {Agent : Type u_1} [Fintype Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) (p : ℝ) =>
{ winners := {i : Agent | p ≤ bids i}, salePrice := fun (x : Agent) => p }

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual fixed-price outcome has winners exactly at bids $\geq p$ and one sale price p , as required by fixed pricing. No bid-dependent price selection is introduced.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.fixedPriceAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.fixedPriceAuction

Source locator: cited publication, lines 97-103

Verbatim paper-source connection

One way to set prices for items in unlimited supply is to estimate consumer utility via market analysis and then set a fixed price. We refer to this method as fixed pricing. Pay-per-view movies are an example of fixed pricing for an unlimited supply market. With perfect knowledge of consumer utilities, optimal fixed pricing maximizes fixed-price revenue by selecting the optimal price at which to sell items. Fixed pricing generally cannot achieve this ideal due to the

Lean-expanded paper semantic target

type:
{Agent : Type u_1} →
[Fintype Agent] →
ℝ → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent
value:
fun {Agent : Type u_1} [Fintype Agent] (p : ℝ) (a : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome a p

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual fixed-price auction applies one externally fixed price to each bid profile. The displayed outcome accepts bids at least that price and charges it to winners, matching source97-103 fixed pricing.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.fixedPriceScaleCondition

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.fixedPriceScaleCondition

Source locator: cited publication, lines 257-265

Verbatim paper-source connection

can make the assumption that the optimal revenue F is significantly larger than h , the highest bid. That is, for some constant α , we assume that $\alpha h \leq F$. With this assumption, optimal fixed pricing sells at least α items. In some cases α is a fixed constant. In other cases, success probability approaches 1 as $\alpha \rightarrow \infty$. For some proofs, this assumption is stronger than what we need and we make the weaker assumption that $\alpha h \leq T$.

Lean-expanded paper semantic target

type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$

value:
 $\text{fun } (\alpha \text{ highest fixedPrice} : \mathbb{R}) \Rightarrow \alpha * \text{highest} \leq \text{fixedPrice}$

Recorded source-to-declaration screening Verdict: matches

Reason: The actual condition is $\alpha \text{ times highest bid} \leq \text{optimal fixed-price revenue}$. This is the first displayed scale inequality in source257-265.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.highestBid

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.highestBid

Source locator: cited publication, lines 130-132

Verbatim paper-source connection

bidders. Without loss of generality, we assume that the lowest bid is one and denote the highest bid by h . Let the total utility T be the sum of all bidders'

Lean-expanded paper semantic target

type:
{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → $\mathbb{R}$

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
Finset.univ.sup' GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue._proof_1 bids

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual highest bid is the finite maximum over a nonempty bid profile, exactly h in source130-132.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.independentHalfSamplingAnalysisLaw

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.independentHalfSamplingAnalysisLaw

Source locator: cited publication, lines 466-469

Verbatim paper-source connection

For the purpose of simplifying our analysis, we will be analyzing a different method of sampling, one that selects a bid to be in the sample independently at random with probability $1/2$. This method of sampling is simpler to analyze, and it does worse than the for5

Lean-expanded paper semantic target

type:
(Agent : Type u_1) → MeasureTheory.Measure (Agent → Bool)

value:
fun (Agent : Type u_1) => AppliedModelingLib.FairCoin.productMeasure Agent

Direct retained dependencies

- [AppliedModelingLib.FairCoin.productMeasure](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual analysis law is the product of Bernoulli($1/2$) coordinate laws, so membership is independently fair for each bidder. This is the distinct simplifying sampling law in source466-469, not the exact-half operational auction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceAuction

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceAuction

Source locator: cited publication, lines 317-335

Verbatim paper-source connection

To get a better understanding of how the optimal single-price auction works, we define the optimal threshold function, $\text{opt}(B)$. This function on a set of bids B returns the fixed price at which items should be sold to achieve revenue F . In the optimal single-price auction, all bid values that are at least $\text{opt}(B)$ will be satisfied at price $\text{opt}(B)$. All other bids will be rejected. That is,

5

Generalized Truthful Auction Mechanisms

By making observations about auctions that encourage utility value bids, in particular the Vickrey auction, we can design general auction mechanisms. We present two general auction mechanisms that facilitate the design of truthful auctions. The first, the bid-independent auction mechanism, is based on the

→ observation that the price that a bid is satisfied at must

be independent from that bid's actual value. The second, the random sampling auction mechanism, is based on the observation that rejected bids can be used to set prices for bids that are to be satisfied.

$\text{opt}(B) = \text{argmax}_{b_i \in B} b_i \cdot (n - i + 1).$

Lean-expanded paper semantic target

type:
{Agent : Type u_1} →
[Fintype Agent] →
[Nonempty Agent] →
GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (a : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceOutcome a

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual optimal single-price auction uses the selected optimal input-bid threshold and fills all bids at least that threshold at that price. This is source317-335.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.optimalThreshold

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.optimalThreshold

Source locator: cited publication, lines 317-335

Verbatim paper-source connection

To get a better understanding of how the optimal single-price auction works, we define the optimal threshold function, $\text{opt}(B)$. This function on a set of bids B returns the fixed price at which items should be sold to achieve revenue F . In the optimal single-price auction, all bid values that are at least $\text{opt}(B)$ will be satisfied at price $\text{opt}(B)$. All other bids will be rejected. That is,

5

Generalized Truthful Auction Mechanisms

By making observations about auctions that encourage utility value bids, in particular the Vickrey auction, we can design general auction mechanisms. We present two general auction mechanisms that facilitate the design of truthful auctions. The first, the bid-independent auction mechanism, is based on the

→ observation that the price that a bid is satisfied at must be independent from that bid's actual value. The

second, the random sampling auction mechanism, is based on the observation that rejected bids can be used to set prices for bids that are to be satisfied.

$\text{opt}(B) = \text{argmax}_{b_i \in B} b_i \cdot (n - i + 1)$.

Lean-expanded paper semantic target

type:

{Agent : Type u_1} → [Fintype Agent] → [Nonempty Agent] → GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → ℝ

value:

```
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
  (GHW01DigitalGoods.SourceDefinitions.optimalThresholdCandidates bids).max'
  (GHW01DigitalGoods.SourceDefinitions.optimalThresholdCandidates_nonempty † bids)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.optimalThresholdCandidates](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual selector takes the largest member of the finite set of input bid prices attaining F . It therefore chooses a genuine argmax input bid as in source 317-335; choosing one maximizing tie representative does not alter the source objective.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.performanceRatio

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.performanceRatio

Source locator: cited publication, lines 239-245

Verbatim paper-source connection

Analogous to on-line algorithm theory, we express auction performance relative to that for optimal untruthful auctions, as ratios R/T or R/F . However, we solve a maximization problem while on-line algorithms solve minimization problems. Thus, positive results, which are lower bounds on R/T or R/F , are expressed using " Ω ". Impossibility results, which are upper bounds on R/T or R/F for any auction in a certain class, are expressed using " O ".

Lean-expanded paper semantic target

type:

$\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$

value:

fun (revenue benchmark : $\mathbb{R}$) => revenue / benchmark

Recorded source-to-declaration screening Verdict: matches

Reason: The actual performance ratio is revenue divided by benchmark, the R/T or R/F quantity in source239-245 on its ordinary nonzero benchmark domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.randomizedRevenueLaw

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.randomizedRevenueLaw

Source locator: cited publication, lines 211-214

Verbatim paper-source connection

outcomes. We use R to denote the auction revenue for a particular auction mechanism and set of bids. R is the sum of all sale prices. For randomized auctions, R is a random variable.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Lean-expanded paper semantic target

```
type:
{Agent : Type u_1} →
  GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent →
    GHW01DigitalGoods.SourceDefinitions.BidProfile Agent → PMF ℝ

value:
fun {Agent : Type u_1} (auction : GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent)
  (bids : GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) =>
  PMF.map GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue (auction bids)
```

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome](#)
- [GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue](#)
- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)
- [GHW01DigitalGoods.SourceDefinitions.RandomizedAuction](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual randomized revenue law pushes the outcome PMF through the winner-price sum. This preserves source211-214 revenue as a random variable under the bound discrete outcome-law convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.scarceSupply

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.scarceSupply

Source locator: cited publication, lines 1623-1625

Verbatim paper-source connection

4 Formally, items are in scarce supply if we have k items and all of the sample and the loss due to having a non-optimal fixed pricing with k or $k + 1$ items would sell all of the items.

Lean-expanded paper semantic target

type:
{Agent : Type u_1} →
[Fintype Agent] → [Nonempty Agent] → (Agent → ℝ) → GHW01DigitalGoods.SourceDefinitions.SupplyCapacity → Prop

value:
fun {Agent : Type u_1} [Fintype Agent] [Nonempty Agent] (bids : Agent → ℝ)
 (capacity : GHW01DigitalGoods.SourceDefinitions.SupplyCapacity) =>
 GHW01DigitalGoods.SourceDefinitions.sellsFullCapacityAtOptimalPrice bids capacity ∧
 GHW01DigitalGoods.SourceDefinitions.sellsFullCapacityAtOptimalPrice bids (capacity + 1)

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.SupplyCapacity](#)
- [GHW01DigitalGoods.SourceDefinitions.sellsFullCapacityAtOptimalPrice](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual predicate requires optimal capped pricing to sell the full capacity for both k and $k+1$. The displayed sales condition uses eligible count $\geq$ capacity, exactly source1623-1625 scarce supply.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHW01DigitalGoods.SourceDefinitions.totalBidValue

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.totalBidValue

Source locator: cited publication, lines 132-134

Verbatim paper-source connection

h. Let the total utility T be the sum of all bidders' utilities. T is an obvious upper bound on the revenue that can be obtained from this set of bidders. Let F

[Next verbatim source excerpt]

does not discourage bidders from bidding their utility value. When considering truthful auctions, we assume that $b_i = u_i$. To enable analysis of auction revenue we define several properties of an input set of bids. As stated in the introduction, T is the sum of all of the bids. An equivalent definition of T is the revenue due to the optimal multi-price untruthful auction, the one that satisfies all bids at their value. The revenue for optimal fixed pricing is F . Note that F can also be

Lean-expanded paper semantic target

type:
 $\{ \text{Agent} : \text{Type } u_1 \} \rightarrow [\text{Fintype Agent}] \rightarrow \text{GHW01DigitalGoods.SourceDefinitions.BidProfile Agent} \rightarrow \mathbb{R}$

value:
 $\text{fun } \{ \text{Agent} : \text{Type } u_1 \} [\text{Fintype Agent}] (\text{bids} : \text{GHW01DigitalGoods.SourceDefinitions.BidProfile Agent}) \Rightarrow \sum i : \text{Agent}, \text{bids } i$

Direct retained dependencies

- [GHW01DigitalGoods.SourceDefinitions.BidProfile](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The actual function is the finite sum of submitted bids, matching the repeated T definition in source228-232. The bound TruthfulReports premise is required only when identifying this with the separate utility-value sum.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GHW01DigitalGoods.SourceDefinitions.totalValueScaleCondition

Source locator: cited publication, lines 257-265

Verbatim paper-source connection

can make the assumption that the optimal revenue F is significantly larger than h , the highest bid. That is, for some constant α , we assume that $\alpha h \leq F$. With this assumption, optimal fixed pricing sells at least α items. In some cases α is a fixed constant. In other cases, success probability approaches 1 as $\alpha \rightarrow \infty$. For some proofs, this assumption is stronger than what we need and we make the weaker assumption that $\alpha h \leq T$.

Lean-expanded paper semantic target

type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$

value:
 $\text{fun } (\alpha \text{ highest total} : \mathbb{R}) \Rightarrow \alpha * \text{highest} \leq \text{total}$

Recorded source-to-declaration screening Verdict: matches
Reason: The actual condition is α times highest bid $\leq$ total value, the weaker scale assumption in source257-265.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 359-370

Verbatim source input

Theorem 4.1 $F \geq T / (2 \log h)$.
Proof. Divide the bids into $\log h$ bins by partitioning the bids at values of powers of two. Thus, each bid is less than a factor of two from any other bid in the same bin. Since the sum of the bids is T and there are $\log h$ bins, then some bin has a sum of at least $T / \log h$. Note that the lowest bid in this bin is at least half of any other bid in the bin. If the optimal single-price auction chose, as its selling price, the price of the lowest bid in this bin, then the contribution of each bid in this bin to the revenue is at least half of the bid's value. Since the bin sums to more than $T / \log h$, this means that the revenue is greater than $T / (2 \log h)$. Thus the optimal fixed pricing can always achieve a revenue of at least $T / (2 \log h)$.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Let $h \geq 2$. For a nonempty finite bid profile whose values all lie in $[1, h]$, its total value T satisfies $T \leq (2 \log_2 h) F$, where F is the optimal fixed-price revenue.

Archival source anchor: cited publication, lines 359-370

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (values : Agent → ℝ) {h : ℝ},
  2 ≤ h →
  (∀ (i : Agent), 1 ≤ values i) →
  (∀ (i : Agent), values i ≤ h) →
  (∀ (highest : ∃ (i : Agent), values i = h),
    have x : Nonempty Agent := GHW01DigitalGoods.PaperInterface.result_theorem4_1Spec._proof_2 values _highest;
    GHW01DigitalGoods.ProofBridge.totalValue values ≤
      2 * Real.logb 2 h * GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values)
```

Retained governing declarations

- [GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark](#)
- [GHW01DigitalGoods.ProofBridge.totalValue](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Fintype Agent
3. parameter: Agent → ℝ
4. parameter: ℝ
5. assumption: 2 ≤ h
6. assumption: ∀ (i : Agent), 1 ≤ values i
7. assumption: ∀ (i : Agent), values i ≤ h
8. assumption: ∃ (i : Agent), values i = h
9. conclusion: GHW01DigitalGoods.ProofBridge.totalValue values ≤ 2 * Real.logb 2 h * GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem4_1

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The archival Theorem 4.1 wording is retained separately. The expanded Spec matches the exact approved corrected target and reports that target as a correction rather than a verbatim SODA claim.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 301-307

Verbatim source input

Corollary 4.2 $F \geq T / (4 \log n)$.
Proof. Let p be the optimal price; clearly $p \geq h/n$.
If one drops all bids with values below p , F does not change and T decreases by at most a factor of two.
After the bids are dropped, the ratio of the highest and the lowest bid values is at most n , yielding the desired result.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Let $n \geq 2$ be the number of bidders. For a nonempty finite nonnegative bid profile with positive maximum value h and every value at most h , $T \leq (4 \log_2 n) F$, where F is the optimal fixed-price revenue.

Archival source anchor: cited publication, lines 301-307

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (values : Agent → ℝ) {h : ℝ},
  (∀ (i : Agent), 0 ≤ values i) →
    0 < h →
      ∀ (hmax : ∃ (i : Agent), values i = h),
        (∀ (i : Agent), values i ≤ h) →
          2 ≤ ↑(Fintype.card Agent) →
            have x : Nonempty Agent := GHW01DigitalGoods.PaperInterface.result_theorem4_1Spec._proof_2 values _hmax;
            GHW01DigitalGoods.ProofBridge.totalValue values ≤
              4 * Real.logb 2 ↑(Fintype.card Agent) * GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values
```

Retained governing declarations

- [GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark](#)
- [GHW01DigitalGoods.ProofBridge.totalValue](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Fintype Agent
3. parameter: Agent → ℝ
4. parameter: ℝ
5. assumption: ∀ (i : Agent), 0 ≤ values i
6. assumption: 0 < h
7. assumption: ∃ (i : Agent), values i = h
8. assumption: ∀ (i : Agent), values i ≤ h
9. assumption: 2 ≤ ↑(Fintype.card Agent)
10. conclusion: GHW01DigitalGoods.ProofBridge.totalValue values ≤ 4 * Real.logb 2 ↑(Fintype.card Agent) * GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_corollary4_2

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The archival Corollary 4.2 wording is retained separately. The expanded Spec matches the exact approved corrected target, including its explicit finite-domain condition, and does not represent that condition as printed in the SODA text.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 428-454

Verbatim source input

Lemma 6.1 Consider a set A and its subset $B \subset A$. Suppose we pick an integer k such that $0 < k < |A|$ and a random subset (sample) $S \subset A$ of size k . Then for $0 < \delta \leq 1$ we have $\Pr[|S \cap B| < (1 - \delta)|B| \cdot k/|A|] < \exp(-|B| \cdot k \delta^2 / (2|A|))$. Proof. We refer to elements of A as points. Note

Random Sampling Optimal that $|S \cap B|$ is the number of sample points in B , and its expected value is $|B| \cdot k/|A|$. Let $p = k/|A|$. Threshold Auction

If instead of selecting a sample of size exactly k we choose each point to be in the sample independently with probability p then the Chernoff bound would yield the lemma.

Let $A = \{a_1, \dots, a_n\}$ and without loss of generality assume that $B = \{a_1, \dots, a_k\}$. We can view the process of selecting S as follows. Consider the elements of A in the order induced by the indices. For each element a_i considered, select the element with probability p_i , where p_i depends on the selections made up to this point.

Let t be the number of points already selected when a_{i+1} is considered. Then $i - t$ is the number of points considered but not selected. Suppose that $t/i < p$.

Then $p_{i+1} > p$.

We conclude that when we select the sample as a random subset of size k , the probability that the number of sample points in B is less than the expected

Expanded PaperInterface specification

```
∀ {α : Type u_1} {A B : Finset α},
  B ⊆ A →
  ∀ {k : ℕ},
    0 < k →
    k < A.card →
    ∀ {δ : ℝ},
      0 < δ →
      δ ≤ 1 →
      (GHW01DigitalGoods.fixedSizeSampleProbability A k fun (sample : Finset α) =>
        ↑(GHW01DigitalGoods.fixedSizeHitCount B sample) < (1 - δ) * ↑B.card * ↑k / ↑A.card) <
        Real.exp (-(↑B.card * ↑k * δ ^ 2 / (2 * ↑A.card))))
```

Retained governing declarations

- [GHW01DigitalGoods.fixedSizeHitCount](#)
- [GHW01DigitalGoods.fixedSizeSampleProbability](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Finset α
3. parameter: Finset α
4. assumption: B ⊆ A
5. parameter: ℕ
6. assumption: 0 < k
7. assumption: k < A.card
8. parameter: ℝ
9. assumption: 0 < δ
10. assumption: δ ≤ 1
11. conclusion: (GHW01DigitalGoods.fixedSizeSampleProbability A k fun (sample : Finset α) =>
  ↑(GHW01DigitalGoods.fixedSizeHitCount B sample) < (1 - δ) * ↑B.card * ↑k / ↑A.card) <
  Real.exp (-(↑B.card * ↑k * δ ^ 2 / (2 * ↑A.card))))
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_lemma6_1_fixed_size_lower_tail

Recorded source-to-Spec screening Verdict: matches

Reason: Source-first review: the Spec states the uniform fixed-size subset lower-tail event and the exponential bound with the source's k , $|A|$, $|B|$, and δ parameters.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 455-547

Verbatim source input

The random sampling optimal threshold auction takes $f = \text{opt}$, the optimal threshold function, in the random sampling auction. Intuitively we use the sample to get an idea for a good threshold value, then we apply that threshold to the remaining bids. The auction samples $m = n/2$ bids at random, computes opt of this sample, and uses this value as a threshold for the non-sample, accepting all bids above this threshold at the threshold value. In this section we show that the expected revenue of this auction is within a constant factor of F , assuming F/h is not too small.

For the purpose of simplifying our analysis, we will be analyzing a different method of sampling, one that selects a bid to be in the sample independently at random with probability $1/2$. This method of sampling is simpler to analyze, and it does worse than the for5

[form-feed in source extraction]

value is smaller than in the case we select each point show that $R \geq F/(2+\alpha)$ with probability that goes to 1 as α goes to infinity, but the convergence is slower for smaller α .

Let R be the revenue of the random sampling optiBy symmetry, the expected revenue of the dualmal threshold auction. The following theorem shows price

↪ variant of the random sampling auction is twice that $R = \Omega(F)$ with probability going to one as α the expected revenue of the original. One can show goes to ∞ .

that the expected revenue of the dual-price auction

Theorem 6.2 Assume $\alpha h \leq F$. Then $R \geq F/6$ is at least $F/(1 + \alpha)$ if α is large enough.

On the other hand, for many input distributions the with probability of at least $1 - e^{-\alpha/36} - 40e^{-\alpha/72}$ threshold value of a sample has a non-zero probability

Proof. Let k be the number of bids satisfied in of being non-optimal for the non-sample. If this is the the optimal single-price solution. Consider the opti- case, the expected value of the revenue in the random mal fixed pricing revenue of the sample, F' . Since sampling auction that samples half of the bids is less we expect $k/2$ of these bids to be in the sample, than $F/2$, and the expected value of the dual price $E[F'] \geq F/2$. Applying Lemma 6.1 with A the set auction is less than F .

of all bids, B the set of bids in the optimal threshold solution on A , and $\delta = 1/3$, we conclude that

Weighted Pairing

$|S \cap B| < |B|/3 = k/3$ with probability at least $1 - e^{-k/36}$

. The assumption $\alpha h \leq F$ implies $k \geq \alpha$.

$1 - e^{-k/36}$

Thus $F' \geq F/3$ with probability at least $1 - e^{-\alpha/36}$. All truthful auctions we introduced so far are either

Let k_s be the number of bids satisfied by optimal single-price or dual-price. In this section we describe fixed pricing on the sample (i.e. the number of bids a multi-price auction. The weighted pairing auction in the sample that are at least $\text{opt}(B')$) and let k_n we present is in the bid-independent class with the be the number of bids in the non-sample that are at function f defined as follows:

least $\text{opt}(B')$. If $F' \geq F/3$ then $k_s \geq \alpha/3$.

$f(B) = b \in B$ w.p. $[U+0002 \text{ control character}] \cdot b'$

Now, assuming that $k_s \geq \alpha/3$, we show that the

$b \in B$

probability that $k_n < k_s/2$ is small. Note that $k_n <$

$k_s/2$ implies that among the top $(3/2)k_s$ bids, at least Thus, to determine if bidder i wins the auction and

k_s are in the sample. Note that the sample and the at what price, pick a bid $b \in B_i$ with probability $\text{pr}_{\text{non-sample}}$ are symmetric: taking a random subset

↪ proportional to the value of b , i.e., $b/(T - b_i)$. This pairs

containing half of the elements in A is equivalent to b_i with b . If $b \leq b_i$, bidder i wins at cost b , othertaking a complement of a random subset of half of wise

↪ i loses. Note that the result of this selection for the elements. We apply Lemma 6.1 with S being i does not affect the auction outcome for the bidder

the non-sample, B being the top $i = (3/2)k_s$ bids, who bid b .

To understand the intuition behind this auction,

and $\delta = 1/3$, and conclude that the probability that

consider

a related random pairing auction. Assume

$k_n < k_s/2 = i/3$ is at most $e^{-i/36}$.

that

n

is

even and pair bidders at random, indepenIf $R < F'/2$ then $k_n < k_s/2$ and thus for some

dent

of

their

bid values. For each pair, conduct a $1i \geq (3/2)\alpha/3 = \alpha/2$ we have $k_n < i/3$. Using the

item

Vickrey

auction, that is, for a pair (b_i, b_j) with

union bound, the probability that this happens is at

b

$<$

b

;

i

loses

and j wins at cost b_i .

i

j

most

∞

Compared

to

the random pairing auction, a bidder

[U+0001 control character]

$e^{-i/36} < 40e^{-\alpha/72}$.
in the weighted pairing auction is less likely to win,
 $i = \alpha/2$
but when a bidder wins they are likely to pay more.
Using the union bound for the probabilities that it turns out that high bidders are still likely to win
 $R \geq F'/2$ and $F' \geq F/3$, we conclude that $R \geq F/6$ the auction, and the benefit of high bidders paying
higher prices outweighs the benefit of more low bid with probability at least $1 - e^{-\alpha/36} - 40e^{-\alpha/72}$.

[Next verbatim source excerpt]

The random sampling optimal threshold auction
takes $f = \text{opt}$, the optimal threshold function, in
the random sampling auction. Intuitively we use the
sample to get an idea for a good threshold value,
then we apply that threshold to the remaining bids.
The auction samples $m = n/2$ bids at random, computes opt of this sample, and uses this value as a
threshold for the non-sample, accepting all bids above
this threshold at the threshold value. In this section
we show that the expected revenue of this auction is
within a constant factor of F , assuming F/h is not
too small.
For the purpose of simplifying our analysis, we will
be analyzing a different method of sampling, one that
selects a bid to be in the sample independently at random with probability $1/2$. This method of sampling
is simpler to analyze, and it does worse than the for5

[form-feed in source extraction]

value is smaller than in the case we select each point show that $R \geq F/(2+\phi)$ with probability that goes to
to be in the sample with probability p .
1 as α goes to infinity, but the convergence is slower
for smaller ϕ .

[Next verbatim source excerpt]

To enable analysis of auction revenue we define several properties of an input set of bids. As stated in
the introduction, T is the sum of all of the bids. An
equivalent definition of T is the revenue due to the
optimal multi-price untruthful auction, the one that
satisfies all bids at their value. The revenue for optimal fixed pricing is F . Note that F can also be
interpreted as the revenue due to the optimal single-price untruthful auction. More discussion of F and
its relation to the optimal single-price untruthful auction is given in Section 4. Other properties that we
use in analysis are ell , the smallest bid value, and h ,
the highest bid value. Because bids can be arbitrarily scaled, we assume, without loss of generality, that
 $\text{ell} = 1$, in which case h is really the ratio of the highest
bid to the lowest bid.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/
SOURCE_CLARIFICATIONS.md).

Expanded PaperInterface specification

```

∀ (m : ℕ) (hm_pos : 0 < m),
  have x : NeZero (m + m) := GHW01DigitalGoods.PaperInterface.result_theorem6_2_random_samplingSpec._proof_3 m hm_pos;
  ∀ (values : Fin (m + m) → ℝ) {alpha : ℕ} {highValue : ℝ},
    0 < highValue →
      (∀ (i : Fin (m + m)), values i ≤ highValue) →
        (∃ (i : Fin (m + m)), values i = highValue) →
          ↑alpha * highValue ≤ GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values →
            1 - Real.exp (-↑alpha / 36) - 40 * Real.exp (-↑alpha / 72) ≤
              GHW01DigitalGoods.pmfEventProbability
                (GHW01DigitalGoods.uniformFixedSizeSampleLaw m
                  (GHW01DigitalGoods.PaperInterface.result_theorem6_2_random_samplingSpec._proof_8 m hm_pos))
            fun (sample : 1 (GHW01DigitalGoods.FixedSizeSampleSpace (Fin (m + m)) m)) =>
              GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values ≤
                6 *
                  GHW01DigitalGoods.ProofBridge.revenue
                    (GHW01DigitalGoods.randomSamplingOptimalThresholdAuction
                      (GHW01DigitalGoods.sampledSideAssignment ↑sample) true 1)
            values

```

Retained governing declarations

- [GHW01DigitalGoods.FixedSizeSampleSpace](#)
- [GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark](#)
- [GHW01DigitalGoods.ProofBridge.revenue](#)
- [GHW01DigitalGoods.pmfEventProbability](#)
- [GHW01DigitalGoods.randomSamplingOptimalThresholdAuction](#)
- [GHW01DigitalGoods.sampledSideAssignment](#)
- [GHW01DigitalGoods.uniformFixedSizeSampleLaw](#)

Lean-elaborated claim roles

```
1. parameter: ℕ
2. assumption: 0 < m
3. conclusion: ∀ (values : Fin (m + m) → ℝ) {alpha : ℕ} {highValue : ℝ},
  0 < highValue →
  (∀ (i : Fin (m + m)), values i ≤ highValue) →
  (∃ (i : Fin (m + m)), values i = highValue) →
  ↑alpha * highValue ≤ GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values →
  1 - Real.exp (- ↑alpha / 36) - 40 * Real.exp (- ↑alpha / 72) ≤
  GHW01DigitalGoods.pmfEventProbability
  (GHW01DigitalGoods.uniformFixedSizeSampleLaw m
   (GHW01DigitalGoods.PaperInterface.result_theorem6_2_random_samplingSpec._proof_8 m _hm_pos))
fun (sample : 1 (GHW01DigitalGoods.FixedSizeSampleSpace (Fin (m + m)) m)) =>
  GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values ≤
  6 *
  GHW01DigitalGoods.ProofBridge.revenue
  (GHW01DigitalGoods.randomSamplingOptimalThresholdAuction
   (GHW01DigitalGoods.sampledSideAssignment ↑sample) true 1)
  values
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem6_2_random_sampling

Recorded source-to-Spec screening Verdict: matches

Reason: Source-first review: under $\alpha^*h \leq F$, the Spec preserves the $F/6$ revenue event and the stated $1 - \exp(-\alpha/36) - 40\exp(-\alpha/72)$ probability bound; exact-half operation and independent-half analysis are explicitly disambiguated.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 563-607

Verbatim source input

Theorem 7.1 If $4h \leq T$, then for the weighted pairing auction $E[R] = \Omega(T / \log h)$.

[U+0002 control character]
[U+0002 control character] S_j^2

We are interested in $E[R] \geq$

$E[R_j] \geq$

$48T$.
Since S_j 's sum up to T , the sum is minimized when

Proof. Partition the n bids into $\log h$ bins as be- all $\log h$ of the S_j 's are equal to $T / \log h$. In this
fore such that bin j contains only bids in the interval case,
[$2^{j-1}, 2^j$]. Recall that an important property of these

$T / 2 (\log h)$

bins is that bids in the same bin are within factor

$E[R] \geq$

$48T \log^2 h$

of two from each other. Let S_j be the sum of the
bids in bin j . Note that the sum of the contents of

bins

contain only one bid is upper bounded by Since $T > T / 2$, we have

[U+0002 control character]log that

h_j

$j=1 \ 2 = 2h - 1 < T / 2$. We will ignore such bins

T

in our analysis below. Consider all bins with two or

$E[R] \geq$

192

$\log h$

more bids, and let T be the sum of bids in these

bins. We have $T > T / 2$.

Thus, $E[R] = \Omega(T / \log h)$.

For each bin j , we look at pairings of bids that

are both in j and we bound the expected revenue

Note that the constant here is $1/192$ which does

due to such pairings. First, the probability that a

not seem too good. However, the analysis was very

bid i in bin j is paired with another bid in bin j is

loose in contributions to R that it considered. In

$(S_j - b_i) / (T - b_i) > S_j / (3T)$, since bin j contains at

practice, this auction has much better constants. See

least two bids and $S_j - b_i \geq S_j / 3$.

Section 10 for details.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Let $h \geq 2$. Under Theorem 7.1's normalized bid and large-market conditions, the operational weighted-pairing auction satisfies $T \leq 576 \log_2(h) W$.

Archival source anchor: cited publication, lines 563-607

Expanded PaperInterface specification

$\forall \{ \text{Agent} : \text{Type } u_1 \} [\text{inst} : \text{Fintype Agent}] (\text{values} : \text{Agent} \rightarrow \mathbb{R}) \{ h \text{ total} : \mathbb{R} \},$

$\text{total} = \text{GHW01DigitalGoods.ProofBridge.totalValue values}$

$2 \leq h \rightarrow$

$(\forall (i : \text{Agent}), 1 \leq \text{values } i) \rightarrow$

$(\forall (i : \text{Agent}), \text{values } i \leq h) \rightarrow$

$(\exists (i : \text{Agent}), \text{values } i = h) \rightarrow$

$4 * h \leq \text{total} \rightarrow \text{total} \leq 576 * \text{Real.logb } 2 \ h * \text{GHW01DigitalGoods.ProofBridge.weightedPairingRevenue values}$

Retained governing declarations

- [GHW01DigitalGoods.ProofBridge.totalValue](#)
- [GHW01DigitalGoods.ProofBridge.weightedPairingRevenue](#)

Lean-elaborated claim roles

1. parameter: $\text{Type } u_1$

2. parameter: Fintype Agent

3. parameter: $\text{Agent} \rightarrow \mathbb{R}$

4. parameter: $\mathbb{R}$

5. parameter: $\mathbb{R}$

6. assumption: $\text{total} = \text{GHW01DigitalGoods.ProofBridge.totalValue values}$

7. assumption: $2 \leq h$

8. assumption: $\forall (i : \text{Agent}), 1 \leq \text{values } i$

9. assumption: $\forall (i : \text{Agent}), \text{values } i \leq h$

10. assumption: $\exists (i : \text{Agent}), \text{values } i = h$

11. assumption: $4 * h \leq \text{total}$

12. conclusion: $\text{total} \leq 576 * \text{Real.logb } 2 \ h * \text{GHW01DigitalGoods.ProofBridge.weightedPairingRevenue values}$

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem7_1_weighted_pairing

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The archival Theorem 7.1 wording is retained separately. The expanded Spec matches the exact approved corrected target for the weighted-pairing lower bound and does not treat its added formal conditions as printed source text.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 615-680

Verbatim source input

We show that the expected revenue of the weighted

√

pairing auction is $\Omega(F / \log h)$, and that this bound is tight in the worst case. This implies that the weighted pairing auction is not competitive.

We use the following notation. We denote the expected revenue of the weighted pairing auction by W .

let $k = \log h$ and $s = \log h$ First we give bound the revenue from below.

√

Theorem 7.2 If $F \geq 2h$, then $W = \Omega(F / \log h)$ and this bound is tight.

$i-1$

$i-1$

$>$

$.$

$2(k-i) + (i-1)$

$2k$

This comes from assuming that all bids below bid i are at value 2^{j-1} and all bids above bid i are at value 2^j . This is the worst it could possibly be. The expected number of bids in bin j that win when paired with other bids from the bin is at least

k

[U+0001 control character]

$i-1$

$i=1$

$2k$

$=$

$k-1$

$.$

4

The smallest bid in bin j has the value of at least Proof. If $F \leq T/s$, then since $W = \Omega(T/k)$ we have $S_j/(2k)$. Let R_j be the revenue generated by bids in $W = \Omega(T/s)$.

bin j being paired with other bids in bin j . We have:

Assume $F > T/s$. Partition the bids over buckets, with bucket i containing bids in the range $[2^i, 2^{i+1})$.

$S_j S_{j-k-1}$

Let M be the set of bids in a bucket with the largest

$.$

$.$

$E[R_j] \geq$

$3T/2k$

4

total bid value and assume that M is defined by the

$(k-1)S_j/2$

value range $[t, 2t)$. We show that the revenue due

$\geq$

$24kT$

only to M is big. The assumption $F \geq 2h$ implies

7

[form-feed in source extraction]

8

that $|M| \geq 2$. (If $|M| = 1$ then $M = \{h\}$ and the previous bucket contains a single bid; in this case $F < 2h$.) Recall that $|M|t = \Omega(F)$. Let $M' = M \setminus M''$.

The probability that an item in M' is paired up

with an item in M'' is $\Omega(T/F) = \Omega(1/s)$, and the

expected revenue is $\Omega(|M'|t/s) = \Omega(F/s)$.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Let $h \geq 2$ and $F \geq 2h$. Under Theorem 7.2's normalized bid conditions, the operational weighted-pairing auction satisfies $F \leq 2304 \sqrt{\log_2 h} W$.

Archival source anchor: cited publication, lines 615-680

Expanded PaperInterface specification

$\forall \{ \text{Agent} : \text{Type } u_1 \} \{ \text{inst} : \text{Fintype Agent} \} \{ \text{values} : \text{Agent} \rightarrow \mathbb{R} \} \{ h \text{ total} : \mathbb{R} \},$
total = GHW01DigitalGoods.ProofBridge.totalValue values $\rightarrow$

$2 \leq h \rightarrow$

$(\forall (i : \text{Agent}), 1 \leq \text{values } i) \rightarrow$

$(\forall (i : \text{Agent}), \text{values } i \leq h) \rightarrow$

$\forall (h_{\text{highest}} : \exists (i : \text{Agent}), \text{values } i = h),$

have $x : \text{Nonempty Agent} := \text{GHW01DigitalGoods.PaperInterface.result_theorem4_1Spec_proof_2 values_h_highest};$

$2 * h \leq \text{GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values} \rightarrow$

$\text{GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values} \leq$

$2304 * \sqrt{\text{Real.logb } 2 h} * \text{GHW01DigitalGoods.ProofBridge.weightedPairingRevenue values}$

Retained governing declarations

- [GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark](#)
- [GHW01DigitalGoods.ProofBridge.totalValue](#)
- [GHW01DigitalGoods.ProofBridge.weightedPairingRevenue](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Fintype Agent
3. parameter: Agent → ℝ
4. parameter: ℝ
5. parameter: ℝ
6. assumption: total = GHW01DigitalGoods.ProofBridge.totalValue values
7. assumption: 2 ≤ h
8. assumption: ∀ (i : Agent), 1 ≤ values i
9. assumption: ∀ (i : Agent), values i ≤ h
10. assumption: ∃ (i : Agent), values i = h
11. conclusion: 2 * h ≤ GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values →
  GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values ≤
  2304 * √(Real.logb 2 h) * GHW01DigitalGoods.ProofBridge.weightedPairingRevenue values
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem7_2_lower_bound

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The archival Theorem 7.2 lower-bound wording is retained separately. The expanded Spec matches the exact approved corrected target and keeps the correction disposition distinct from the SODA presentation.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 626-720

Verbatim source input

Theorem 7.2 If $F \geq 2h$, then $W = \Omega(F / \log h)$ and this bound is tight.

$i-1$
 $i-1$
>
.
 $2(k-i) + (i-1)$
 $2k$
This comes from assuming that all bids below bid i are at value $2j-1$ and all bids above bid i are at value $2j$. This is the worst it could possibly be. The expected number of bids in bin j that win when paired with other bids from the bin is at least
 k
[U+0001 control character]
 $i-1$
 $i=1$

$2k$

=

$k-1$

.

4

The smallest bid in bin j has the value of at least $2j-1$. Proof. If $F \leq T/s$, then since $W = \Omega(T/k)$ we have $S_j/(2k)$. Let R_j be the revenue generated by bids in $W = \Omega(T/s)$.

bin j being paired with other bids in bin j . We have:

Assume $F > T/s$. Partition the bids over buckets, with bucket i containing bids in the range $[2i, 2i+1)$.

$S_j S_{j-k-1}$

Let M be the set of bids in a bucket with the largest

.

.

$E[R_j] \geq$

$3T/2k$

4

total bid value and assume that M is defined by the

$(k-1)S_j/2$

value range $[t, 2t)$. We show that the revenue due

$\geq$

$24kT$

only to M is big. The assumption $F \geq 2h$ implies

7

[form-feed in source extraction]

8

that $|M| \geq 2$. (If $|M| = 1$ then $M = \{h\}$ and the previous bucket contains a single bid; in this case $F < 2h$.) Recall that $|M|t = \Omega(F)$. Let M' be the highest $\lfloor |M|/2 \rfloor$ elements of M and let $M'' = M \setminus M'$.

The probability that an item in M' is paired up with an item in M'' is $\Omega(T/F) = \Omega(1/s)$, and the expected revenue is $\Omega(|M'|t/s) = \Omega(F/s)$.

The above argument considers only contributions of items in M . Somewhat surprisingly, the resulting bound is tight up to a constant factor. We give an example where the expected revenue W of the weighted pairing auction is $\Omega(T/s)$. The bid values are $2i$ for $i = 1, \dots, k$. For each $i = 1, \dots, k-1$ the number of bids of this value is $2k-i$ and the total value of such bids is $2k$. For the last value, $2k$, there are s bids of this value for the total of $s2k = F$. Note that

$T = 2k(s + k - 1)$.

We show that $W = \Omega(2k)$ by first showing that the expected contribution of all bids of value less than $2k$ to W is $\Omega(2k)$ and then showing that the contribution of all bids of value $2k$ is $\Omega(2k)$.

For $i < k$, the probability that a bid of value $2i$

wins is less than

i

$i2k$

$i2k - 2i$

$\leq$

.

$\leq$

i

$T - 2$

T

k

The expected contribution of a winning bid is less

than

i

$1[U+0001 control character] j$

1

$2 < 2i+1$.

$\sum_{j=1}^i$

i

.

An Upper Bound

A natural question to ask is if the bound of Theorem 7.1 can be improved upon. We already know that no single-price auction can do better than F . In this section we prove that no truthful multi-price auction can have an expected revenue greater than F . This result contrasts that of Section 6. In that

Expanded PaperInterface specification

$\forall (s : \mathbb{N}) (hs_two : 2 \leq s),$
 $GHW01DigitalGoods.ProofBridge.weightedPairingRevenue (AppliedModelingLib.Auction.ghwTightValue (s * s + 2) s) \leq$
 $6 / \sqrt{(\text{Real.logb } 2 (2 ^ (s * s + 2)))} *$
 $GHW01DigitalGoods.ProofBridge.ghwTightFixedPriceBenchmarkValue (s * s + 2) s hs_two$

Retained governing declarations

- [AppliedModelingLib.Auction.GhwTightAgent](#)
- [AppliedModelingLib.Auction.GhwTightLowerAgent](#)
- [AppliedModelingLib.Auction.ghwTightValue](#)
- [GHW01DigitalGoods.ProofBridge.ghwTightFixedPriceBenchmarkValue](#)
- [GHW01DigitalGoods.ProofBridge.weightedPairingRevenue](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{N}$
 2. assumption: $2 \leq s$
 3. conclusion: $GHW01DigitalGoods.ProofBridge.weightedPairingRevenue (AppliedModelingLib.Auction.ghwTightValue (s * s + 2) s) \leq$
 $6 / \sqrt{(\text{Real.logb } 2 (2 ^ (s * s + 2)))} *$
 $GHW01DigitalGoods.ProofBridge.ghwTightFixedPriceBenchmarkValue (s * s + 2) s hs_two$

Lean proof endpoint (built separately)

$GHW01DigitalGoods.PaperInterface.result_theorem7_2_tightness$

Recorded source-to-Spec screening Verdict: matches

Reason: Source-first review: the expanded Spec retains the displayed two-level witness family and its $O(F/\sqrt{\log h})$ weighted-pairing upper bound, which is the source tightness conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 746-795

Verbatim source input

Now we show that in a truthful auction, probabilities of winning are monotone functions of bid values.
Lemma 8.1 Suppose in a truthful auction $b_i < b_j$.
Then $p_i \leq p_j$.

This all bids of value $2i$ contribute less than $2k+1/k$.
Summing over $i = 1, \dots, k-1$, we conclude that
the expected contribution of the corresponding bids
is less than $2 \cdot 2k$.

Next we consider the bids of value $2k$. The probability that such a bid wins is at least one and the
expected revenue of a winning bid is at least

Proof. Suppose that i and j have the same utility
equal to b_i . Since the auction is truthful, i 's gain is
at least as big as j 's, thus
 $p_i(b_i - c_i) \geq p_j(b_i - c_j)$.
Similarly, if both utilities are b_j , we get

$k-1$
 s
 $s+1$ k
1 [U+0001 control character] i
 $2k$
 k
 $2 +$
 $2 =$
 $.$
 $2 <$
 $k+s$
 $k + s$ $i=1$
 $k+s$
 s

$p_j(b_j - c_j) \geq p_i(b_j - c_i)$.

Thus the total contribution of such bids is less than Adding the above inequalities and simplifying, we get
 $2k$.
 $p_i b + p_j b \geq p_i b + p_j b$.

j
 j
 j i
 i j

Although the weighted pairing auction is not competitive in the worst case, it is only a factor of $\log h$
petitive in the worst case, it is only a factor of $\log h$
 $p_j(b_j - b_i) \geq p_i(b_j - b_i)$.
away from being competitive, is quite different from
our random sampling auctions, and performs reasonably well. Since $b_j > b_i$, we have $p_j \geq p_i$.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For finite offer-price support, bidder values, prices, and bidder-specific offer laws satisfying the journal Definition 8.1 condition that $b_i \leq b_j$ and $x \leq b_i$
 $\Rightarrow$ imply $\text{CDF}_i(x) \leq \text{CDF}_j(x)$, every strict inequality $b_i < b_j$ implies that bidder i 's probability of receiving an offer at most b_i is at most bidder j 's
 $\Rightarrow$ probability of receiving an offer at most b_j .

Archival source anchor: cited publication, lines 746-795

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Price : Type u_2} [inst : Fintype Price] (values : Agent → ℝ) (price : Price → ℝ)
  (offerLaw : Agent → PMF Price),
  (∀ (i j : Agent),
    values i ≤ values j →
      ∀ t ≤ values i,
        (AppliedModelingLib.pmfProb (offerLaw i) fun (p : Price) => price p ≤ t) ≤
          AppliedModelingLib.pmfProb (offerLaw j) fun (p : Price) => price p ≤ t) →
  ∀ (i j : Agent),
    values i < values j →
      (AppliedModelingLib.pmfProb (offerLaw i) fun (p : Price) => price p ≤ values i) ≤
        AppliedModelingLib.pmfProb (offerLaw j) fun (p : Price) => price p ≤ values j
```

Retained governing declarations

- [AppliedModelingLib.pmfProb](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: Fintype Price

```

4. parameter: Agent → ℝ
5. parameter: Price → ℝ
6. parameter: Agent → PMF Price
7. assumption: ∀ (i j : Agent),
  values i ≤ values j →
  ∀ t ≤ values i,
    (AppliedModelingLib.pmfProb (offerLaw i) fun (p : Price) => price p ≤ t) ≤
    AppliedModelingLib.pmfProb (offerLaw j) fun (p : Price) => price p ≤ t
8. parameter: Agent
9. parameter: Agent
10. assumption: values i < values j
11. conclusion: (AppliedModelingLib.pmfProb (offerLaw i) fun (p : Price) => price p ≤ values i) ≤
  AppliedModelingLib.pmfProb (offerLaw j) fun (p : Price) => price p ≤ values j

```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_lemma8_1_monotone_allocation

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The archival Lemma 8.1 text is retained separately. The expanded Spec matches the exact approved journal-condition corrected target for monotone allocation and does not claim that the additional condition is present in the SODA statement.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 833-971

Verbatim source input

Theorem 8.2 For any truthful auction, $E[R] \leq F$.

$j=1$

$j=1$

$g_i \geq p_{i-1} (b_i - b_{i-1} + b_{i-1} - c_{i-1})$

$= p_n b_n +$

$= p_{i-1} (b_i - b_{i-1}) + p_{i-1} (b_{i-1} - c_{i-1})$

$= p_{i-1} (b_i - b_{i-1}) + g_{i-1}.$

$g_i \geq$

$p_j [b_j - (b_{j+1} - b_j)(n - j)].$

$n-1$

[U+0001 control character]

$p_j [b_j (n - j + 1) - b_{j+1} (n - j)].$

$j=1$

Regrouping

We can recursively expand g_{i-1} in the same way until we get to g_1 which is 0 because all bids are satisfied at value at least 1, and get

$i-1$

[U+0001 control character]

$n-1$

[U+0001 control character]

$E[R] \leq p_n b_n +$

$j=1$

(3) Now, let $V_j = b_j (n - j + 1)$. Intuitively, this is the revenue attained by using b_j as the sale price in fixed pricing. Note that $V_j \leq F$.

Now let R_i be the total expected revenue from bidder i . That is

$n-1$

[U+0001 control character]

$E[R] \leq p_n V_n +$

$p_j (V_j - V_{j+1}).$

$R_i = p_i c_i.$

$j=1$

$j=1$

$p_j (b_{j+1} - b_j).$

We can rearrange equation (1) as $p_i c_i = p_i b_i - g_i$ and Rearrange this sum to sum over V_j instead of p_j and get

for symmetry, define $p_0 = 0$.

$R_i = p_i b_i - g_i.$

$E[R] \leq$

Using equation (3) we get

$R_i \leq p_i b_i -$

$i-1$

[U+0001 control character]

$j=1$

n

[U+0001 control character]

$i=1$

$\leq$

n

[U+0001 control character]

$p_j (b_{j+1} - b_j).$

$E[R] \leq F$

$i=1$

$p_i b_i -$

$i=1$

[U+F8EE private-use glyph]

[U+F8F0 private-use glyph]

$i-1$

[U+0001 control character]

$j=1$
 $(p_j - p_{j-1}) \vee j$.
 $\sum_{j=1}^n$
 $[U+0001 \text{ control character}]$
 $j=1$
 $(p_j - p_{j-1})$
 This sum telescopes to $p_n - p_0$, but since $p_0 = 0$, we have
 R_i
 $\sum_{i=1}^n$
 $[U+0001 \text{ control character}]$
 $j=1$
 But, $\forall j \leq F$ and by Lemma 8.1, $p_j - p_{j-1}$ is nonnegative.

Looking at the sum of the R_i 's, we see that the first term is mostly canceled by the summation term and we get a telescoping effect.
 $E[R] =$

$\sum_{i=1}^n$
 $[U+0001 \text{ control character}]$
 $E[R] \leq p_n F \leq F$.
 $[U+F8F9 \text{ private-use glyph}]$
 $p_j (b_{j+1} - b_j) [U+F8FB \text{ private-use glyph}]$.
 $\frac{9}{9}$

$[form-feed in source extraction]$
 $\frac{9}{9}$

Deterministic Auctions

optimal threshold auction on this input family has revenue of $R = F / h$.
 We have shown several randomized auctions with an
 Next we generalize this result.
 $\Omega(F)$ expected performance under the assumption that $q_h \leq F$ and α is big enough. In this section we study deterministic auctions.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For a randomized offer auction on a finite bidder set with nonnegative bid values and nonnegative real offer prices, with arbitrary bidderwise probability
 $\hookrightarrow$ laws, suppose that whenever $b_i \leq b_j$ and $x \leq b_i$, the probability that bidder i wins at a price at most x is at most the corresponding probability for
 $\hookrightarrow$ bidder j . Then its expected revenue on every bid vector is at most the optimal fixed-price revenue F of that vector.

Archival source anchor: cited publication, lines 833-833

Expanded PaperInterface specification

$\forall \{ \text{Agent} : \text{Type } u_1 \} [\text{inst} : \text{Fintype Agent}] [\text{inst}_1 : \text{Nonempty Agent}] (\text{values} : \text{Agent} \rightarrow \mathbb{R})$
 $(\text{offerLaw} : \text{Agent} \rightarrow \text{MeasureTheory.Measure } \mathbb{R}),$
 $(\forall (i : \text{Agent}), \text{MeasureTheory.IsProbabilityMeasure } (\text{offerLaw } i)) \rightarrow$
 $(\forall (i : \text{Agent}), 0 \leq \text{values } i) \rightarrow$
 $(\forall (i : \text{Agent}), (\text{offerLaw } i) (\text{Set.lit } 0) = 0) \rightarrow$
 $(\forall (i, j : \text{Agent}),$
 $\text{values } i \leq \text{values } j \rightarrow$
 $\forall t \leq \text{values } i, \uparrow(\text{ProbabilityTheory.cdf } (\text{offerLaw } i)) t \leq \uparrow(\text{ProbabilityTheory.cdf } (\text{offerLaw } j)) t) \rightarrow$
 $\text{GHW01DigitalGoods.paper_theorem8_2_continuous_marginal_expected_revenue values offerLaw} \leq$
 $\text{GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values}$

Retained governing declarations

- [GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark](#)
- [GHW01DigitalGoods.paper_theorem8_2_continuous_marginal_expected_revenue](#)

Lean-elaborated claim roles

1. parameter: $\text{Type } u_1$
2. parameter: Fintype Agent
3. assumption: Nonempty Agent
4. parameter: $\text{Agent} \rightarrow \mathbb{R}$
5. parameter: $\text{Agent} \rightarrow \text{MeasureTheory.Measure } \mathbb{R}$
6. assumption: $\forall (i : \text{Agent}), \text{MeasureTheory.IsProbabilityMeasure } (\text{offerLaw } i)$
7. assumption: $\forall (i : \text{Agent}), 0 \leq \text{values } i$
8. assumption: $\forall (i : \text{Agent}), (\text{offerLaw } i) (\text{Set.lit } 0) = 0$
9. assumption: $\forall (i, j : \text{Agent}),$
 $\text{values } i \leq \text{values } j \rightarrow$
 $\forall t \leq \text{values } i, \uparrow(\text{ProbabilityTheory.cdf } (\text{offerLaw } i)) t \leq \uparrow(\text{ProbabilityTheory.cdf } (\text{offerLaw } j)) t$
10. conclusion: $\text{GHW01DigitalGoods.paper_theorem8_2_continuous_marginal_expected_revenue values offerLaw} \leq$
 $\text{GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values}$

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem8_2_journal_revenue_upper_bound

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: Compared the approval-pinned journal Definition 8.1/Theorem 8.2 target with the expanded Spec and its displayed revenue and benchmark definitions. The Spec quantifies a finite bidder profile with nonnegative real values and arbitrary bidderwise probability measures supported on nonnegative real offers, assumes exactly the Definition 8.1 CDF inequality for values $i \leq \text{values } j$ and $t \leq \text{values } i$, and concludes that the sum of expected accepted offers is at most the one-price fixed-price benchmark. On this finite nonnegative profile domain, the displayed candidate-price maximization is the stated optimal fixed-price revenue F . This is the approved journal monotone-offer target, expressly not the archival SODA truthfulness-only claim.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 979-1090

Verbatim source input

truthful deterministic bid-independent auction has Theorem 9.1 For any truthful deterministic bid-independent auction $O(F/h)$ worst-case performance even for $\alpha h \leq F$.

independent auction and any constant α , there exists

Then we show that any deterministic truthful auction an input for which $R/F = O(1/h)$ and $\alpha h \leq F$.

is bid-independent. This implies an upper bound on

worst-case performance for all truthful deterministic Proof. Let f be the (deterministic) function that

defines the auction. Consider an input with n_h bids

at value h and n_{ℓ} bids at value 1. Restricted to such

inputs, f is a function of n_h and n_{ℓ} . Note that we

9.1 Deterministic Optimal Threshold can assume, without loss of generality, that f takes on

Auction

only two values: 1, in which case all bids are satisfied

at the price of 1, and h , in which case the high bids

The deterministic optimal threshold auction is the

are satisfied at price h . Other values of f lead to

bid-independent auction with $f = \text{opt}$, the optimal

smaller revenues.

threshold function defined in Section 4. The only dif We wish to find an input family such that the

ference between the deterministic optimal threshold

bid-independent

auction with function f has revenue

auction and the optimal fixed pricing mechanism is

$O(F$

$/h)$.

To

obtain

this family, we chose n_h and n_{ℓ} in

that the former uses threshold $\text{opt}(B_i)$ for bidder i

such

a

way

that

f

$(n$

$h - 1, n_{\ell}) = 1$, $f(n_h, n_{\ell} - 1) = h$,

and the latter uses $\text{opt}(B)$. Recall that B_i and B

$>$

α

(implying

$\alpha h \leq F$). For such an input,

and

n

h

only differ in that b_i is not in B_i . From this, we

R

$=$

n

and

F

$\geq$

$h n$

so $R/F \leq 1/h$. Our goal now

h

h

might also expect that for large n with suitable conis,

given

a

deterministic

f

, to find values of n_h and

straints on h , the the deterministic optimal threshold

that

have

the

above

properties.

n

ℓ

auction would perform to within a constant fraction

Consider the n_h, n_{ℓ} plane. For a fixed m look at

of F . However, this is not the case as the following

the line $n_h = k$ and $n_{\ell} = m - k$, and consider the line

example illustrates.

The input that shows that the deterministic opti- segment connecting $(0, m)$ and $(m, 0)$. We need to

mal threshold auction performance is $O(F/h)$ is one find a value of k with $k > \alpha$ where, when k increases

with r bids at value h and $q = (h - 1)r - 1$ bids by one, f changes from 1 to h .

Set $m = h/2\alpha$. Assume that $f(\alpha, m - \alpha) = 1$. As

at value 1. On this input, the optimal single-price

auction takes r high bids at price h for a revenue of we increase k from $k = \alpha$ the value of $f(k, m - k)$

$F = hr$. Note that the second-best threshold would must change from 1 to h because for $k = m$ we have

take all n bids at price 1 for revenue of $hr - 1$. The $f(k, m - k) = f(m, 0) = h$. Thus it must be at some

*

*

*

*

*

deterministic optimal threshold auction takes all the k that $f(k, m - k) = h$ and $f(k - 1, m - k + 1) = 1$.

*

*

high bids at value 1 and none of the low bids. This If we now choose $n_h = k$ and $n_{\ell} = m - k + 1$, we

yields a revenue of $R = r$. None of the low bids are satisfy our criteria that $n_h > \alpha$ and that $f(n_h - 1, n_{\text{ell}}) = 1$ and $f(n_h, n_{\text{ell}} - 1) = h$. Thus, $R/F =$ a low bid from the optimal threshold auction does $O(1/h)$. not change the threshold). The high bids are taken Suppose now that our assumption that $f(\alpha, m - 1) = 1$ is false and instead it is h . Then for $n_h = k = m - 1$ at value 1 because removing one h causes the optimal threshold to switch to 1. Thus, the deterministic α and $n_{\text{ell}} = m - k + 1$ we have $F = m + 1 = h/2 \alpha + 1$

[form-feed in source extraction]
value b' would be better off bidding b'' and paying only v . Since the auction is truthful, it must be that $g(b') = v$.

and $R \leq h\alpha$ so
 $R/F \leq h/2h\alpha$
 $\alpha + 1 \leq h/2h\alpha$
 $2\alpha = h/1$

Thus for any bid-independent auction with deterministic function f there exists an input family such that $R = O(F/h)$.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Expanded PaperInterface specification

```

∀ (rule : GHW01DigitalGoods.PaperBidIndependentBoundaryRule) {highValue : ℕ} {alpha : ℝ},
  2 ≤ highValue →
  0 < alpha →
  ∃ (highCount : ℕ) (lowCount : ℕ),
    0 < highCount ∧
    GHW01DigitalGoods.twoValueBidIndependentBoundaryRevenue rule highValue highCount lowCount /
      AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount ≤
      1 / ↑highValue ∧
      ↑highValue * alpha ≤ AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount

```

Retained governing declarations

- [AppliedModelingLib.Auction.twoValueFixedPriceBenchmark](#)
- [GHW01DigitalGoods.PaperBidIndependentBoundaryRule](#)
- [GHW01DigitalGoods.twoValueBidIndependentBoundaryRevenue](#)

Lean-elaborated claim roles

```

1. parameter: GHW01DigitalGoods.PaperBidIndependentBoundaryRule
2. parameter: ℕ
3. parameter: ℝ
4. assumption: 2 ≤ highValue
5. assumption: 0 < alpha
6. conclusion: ∃ (highCount : ℕ) (lowCount : ℕ),
  0 < highCount ∧
  GHW01DigitalGoods.twoValueBidIndependentBoundaryRevenue rule highValue highCount lowCount /
    AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount ≤
    1 / ↑highValue ∧
    ↑highValue * alpha ≤ AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount

```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem9_1_bid_independent_lower_bound

Recorded source-to-Spec screening Verdict: matches

Reason: Source-first review: the Spec captures the source binary witness construction, attained integer high value, positive high-bid count, explicit $1/H$ ratio, and $\alpha \cdot H \leq F$ scale condition for every truthful bid-independent rule.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 1105-1128

Verbatim source input

Lemma 9.2 Any truthful deterministic auction is bid-independent.
Proof. Let $B_i = B \setminus \{i\}$ as before, and let B_{ix} be the set of bids with b_i replaced with value x . Let A be a truthful deterministic auction. Let $A(B)$ denote the result of running A on set B and let $A_i(B)$ denote the result for bidder i (∞ if i is rejected). Consider function g defined as $g(x) = A_i(B_{ix})$. Here g is clearly a function of A and B_i . We show that g is ∞ everywhere except for some region (v, ∞) , $[v, \infty)$, or $\{\}$ where it has value v . This would imply that A is bid-independent. If $g(x) = \infty$ for all x then we have the empty interval, $\{\}$. Otherwise, let $b = \inf\{x : g(x) [U+0016 control character] = \infty\}$ and let $v = \min x(g(x))$. We now show that $b = v$ and for all $b' > b$, $g(b') = v$.
1. $v = b$
First we show that $v \leq b$. This is simple to see because $g(x)$ is defined so that either $g(x)$ is ∞ (and a bid value of x will lose) or $g(x) \leq x$ (and a bid of value x will win at price $g(x)$ which is necessarily at most x).
Now we show that $v \geq b$. Assume for a contradiction that there exists a b' such that $v < b' < b$. This auction is not truthful because a bidder with utility value b' would be better off bidding greater than b , win the auction and pay price v which is less than b' . Thus, $v = b$.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} (M : AppliedModelingLib.Auction.DigitalGoodsAuction Agent),
  GHW01DigitalGoods.ProofBridge.truthful M →
    M.IndividuallyRational →
      M.NoPositiveTransfers →
        M.BinaryAllocation →
          ∀ (bids : Agent → ℝ) (i : Agent),
            AppliedModelingLib.Auction.DeterministicOfferBidIndependent
              (AppliedModelingLib.Auction.deterministicAuctionOffer M bids i)
```

Retained governing declarations

- [AppliedModelingLib.Auction.DeterministicOfferBidIndependent](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.BinaryAllocation](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.IndividuallyRational](#)
- [AppliedModelingLib.Auction.DigitalGoodsAuction.NoPositiveTransfers](#)
- [AppliedModelingLib.Auction.deterministicAuctionOffer](#)
- [GHW01DigitalGoods.ProofBridge.truthful](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: AppliedModelingLib.Auction.DigitalGoodsAuction Agent
3. assumption: GHW01DigitalGoods.ProofBridge.truthful M
4. assumption: M.IndividuallyRational
5. assumption: M.NoPositiveTransfers
6. assumption: M.BinaryAllocation
7. parameter: Agent → ℝ
8. parameter: Agent
9. conclusion: AppliedModelingLib.Auction.DeterministicOfferBidIndependent
  (AppliedModelingLib.Auction.deterministicAuctionOffer M bids i)
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_lemma9_2_bid_independence

Recorded source-to-Spec screening Verdict: matches

Reason: Source-first review: the deterministic truthful auction receives a bidder-specific critical offer determined by the other bids, exactly the source bid-independence conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 1100-1106

Verbatim source input

Truthful Deterministic Auctions Theorem 9.3 For any truthful deterministic auction and any constant α , there exists an input for are Bid-Independent

The following lemma allows us to extend the result of Theorem 9.1 to arbitrary deterministic auctions. Lemma 9.2 Any truthful deterministic auction is bid-independent.

[Next verbatim source excerpt]

which $R/F = O(1/h)$ and $\alpha h \leq F$.

Settled source reading The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE_CLARIFICATIONS.md).

Expanded PaperInterface specification

```

∀ {highValue : ℕ} {alpha : ℝ}
  (model : GHW01DigitalGoods.assumption_theorem9_3_primitive_set_of_bids_deterministic_source_model highValue),
  2 ≤ highValue →
  0 < alpha →
  ∃ (highCount : ℕ) (lowCount : ℕ),
    0 < highCount ∧
    GHW01DigitalGoods.ProofBridge.revenue (model.auctionFamily highCount lowCount)
      (AppliedModelingLib.Auction.twoValueBidProfile highValue highCount lowCount) /
      AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount ≤
      1 / ↑highValue ∧
      ↑highValue * alpha ≤ AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount

```

Retained governing declarations

- [AppliedModelingLib.Auction.TwoValueAgent](#)
- [AppliedModelingLib.Auction.twoValueBidProfile](#)
- [AppliedModelingLib.Auction.twoValueFixedPriceBenchmark](#)
- [GHW01DigitalGoods.ProofBridge.revenue](#)
- [GHW01DigitalGoods.Theorem93.PaperTheorem93PrimitiveSetOfBidsDeterministicSourceModel](#)
- [GHW01DigitalGoods.assumption_theorem9_3_primitive_set_of_bids_deterministic_source_model](#)

Lean-elaborated claim roles

```

1. parameter: ℕ
2. parameter: ℝ
3. parameter: GHW01DigitalGoods.assumption_theorem9_3_primitive_set_of_bids_deterministic_source_model highValue
4. assumption: 2 ≤ highValue
5. assumption: 0 < alpha
6. conclusion: ∃ (highCount : ℕ) (lowCount : ℕ),
  0 < highCount ∧
  GHW01DigitalGoods.ProofBridge.revenue (model.auctionFamily highCount lowCount)
    (AppliedModelingLib.Auction.twoValueBidProfile highValue highCount lowCount) /
    AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount ≤
    1 / ↑highValue ∧
    ↑highValue * alpha ≤ AppliedModelingLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount

```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem9_3_deterministic_lower_bound

Recorded source-to-Spec screening Verdict: matches

Reason: Source-first review: the Spec extends the Theorem 9.1 binary witness conclusion to the explicit truthful deterministic source model; attained high value and sale-price conventions remain visible rather than being inferred from the theorem name.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation